



Supervised Learning: Classification

Making Machines That Can Tell Things Apart



Topic

Classification Algorithms in Machine Learning



Dataset

Breast Cancer Wisconsin Diagnostic Dataset



Tools

Python · scikit-learn · pandas · matplotlib

*Presented by: **Dr. Anilkumar V. Brahmane** / Computer Engineering, Sanjivani College of Engineering*

What is Classification?

Classification is a **supervised machine learning technique** where the model is trained to predict the **class label** of a given input. It finds a **decision boundary** that divides the dataset into distinct classes.

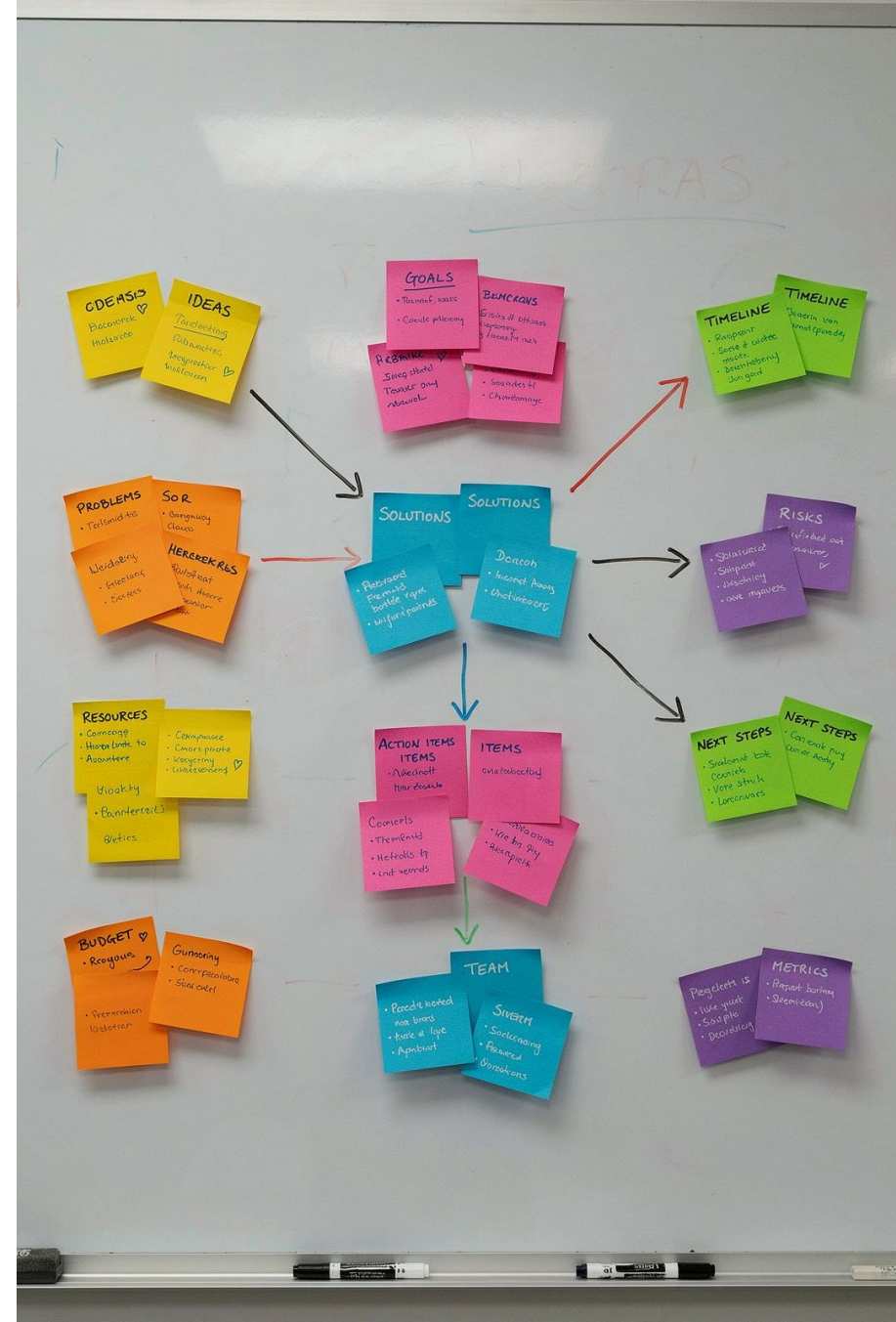
Four Types of Classification

- **Binary** — Two outcomes (Yes/No, Spam/Not Spam)
- **Multi-class** — Multiple categories (e.g., dog, cat, bird)
- **Multi-label** — Multiple labels per instance
- **Imbalanced** — Unequal class distribution

Real-World Applications

- **Healthcare:** Disease diagnosis from imaging data
- **Finance:** Fraud detection & credit scoring
- **Marketing:** Customer segmentation
- **Retail:** Product recommendations
- **Manufacturing:** Quality control & fault detection

— Dr. Anilkumar V. Brahmane | Computer Engineering, Sanjivani College of Engineering —



Classification Theory: The Core Concepts

Understanding the theoretical underpinnings of classification is crucial for building effective machine learning models. This section delves into the fundamental concepts that drive how models learn to distinguish between different classes.

Hypothesis Function $h(x)$

This function maps features (x) to a predicted output class. For a binary classifier, $h(x)$ might output a probability, which is then thresholded to assign a class label.

Feature Space

The multi-dimensional space defined by the input features. Each data point is represented as a vector in this space, and the goal is to find boundaries that separate different classes.

Class Boundaries

These are the decision surfaces or lines that a classifier learns to draw in the feature space, effectively partitioning it into regions corresponding to different classes.

Linear vs. Non-linear Separability

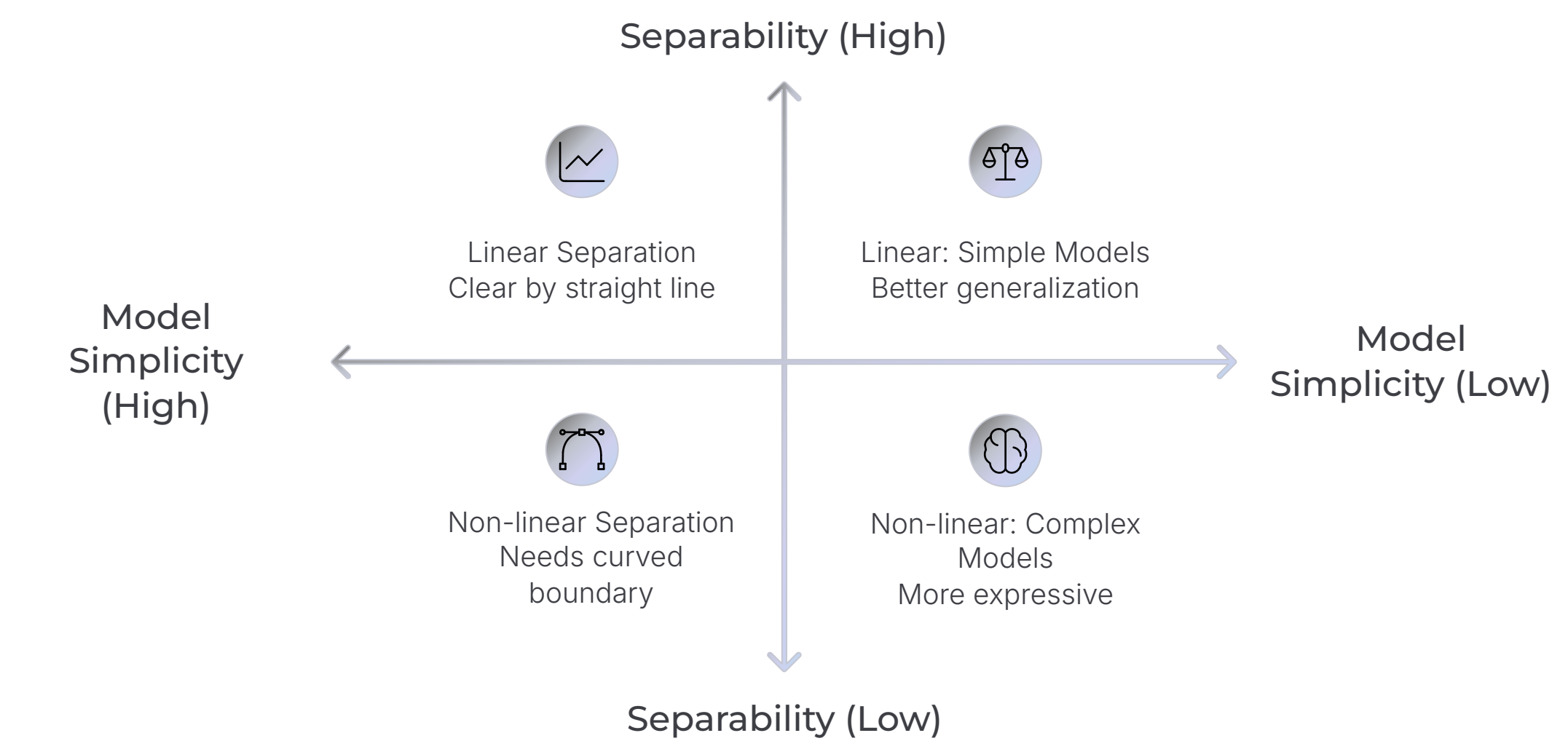
A key concept in classification is whether classes in the feature space can be separated by a straight line (or hyperplane in higher dimensions) or if more complex boundaries are required.

Linearly Separable Data

If a single straight line can perfectly divide the data points belonging to different classes, the data is linearly separable. Models like Logistic Regression and Linear SVM are designed for such scenarios.

Non-linearly Separable Data

When a straight line cannot effectively separate classes, the data is non-linearly separable. This often requires more sophisticated models that can learn curved or complex decision boundaries, such as kernel SVMs or Neural Networks.



This diagram illustrates how data points can be separated, either by a simple straight line (left) or by a more complex, curved boundary (right).

Logistic Regression: Deep Theory

Logistic Regression extends linear regression to classification by modeling the probability of an event belonging to a specific class. This section delves into its mathematical foundations, from probability transformation to parameter optimization.

From Linear to Probabilistic

For classification, we aim to predict the probability $P(Y = 1|X)$. The **odds ratio** is the probability of success to the probability of failure:

$$\text{Odds} = \frac{P(Y = 1|X)}{P(Y = 0|X)} = \frac{p}{1 - p}$$

Applying the logarithm transforms these odds into **log-odds**, or the **logit function**, which can range from $-\infty$ to ∞ , making it suitable for linear modeling:

$$\text{logit}(p) = \log\left(\frac{p}{1 - p}\right)$$

The Sigmoid Function

We assume the log-odds are a linear combination of features: $\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n = X\beta$. Exponentiating both sides and solving for p yields the **sigmoid function** $\sigma(z)$, which squashes any real-valued input z into a probability between 0 and 1:

$$p = \frac{1}{1 + e^{-X\beta}} = \sigma(X\beta)$$

Parameter Optimization: MLE and Gradient Descent

Unlike linear regression which uses Mean Squared Error, Logistic Regression uses **Maximum Likelihood Estimation (MLE)** to find the optimal parameters β . MLE aims to maximize the likelihood of observing the training data given the model's parameters. For m samples, the likelihood function is:

$$L(\beta) = \prod_{i=1}^m p(y_i|x_i; \beta)$$

To simplify, we work with the log-likelihood function. Maximizing this is equivalent to minimizing the negative log-likelihood (our cost function). This optimization is typically done using **Gradient Descent**, an iterative algorithm that adjusts β in the direction opposite to the gradient of the cost function:

$$\beta := \beta - \alpha \nabla_{\beta} J(\beta)$$

Logistic Regression: Regularization & Multi-class Extension

Regularization helps prevent overfitting in models by penalizing large coefficients. For multi-class scenarios, Logistic Regression adapts using strategies like One-vs-Rest or Softmax.

Regularization: Preventing Overfitting

Overfitting occurs when a model learns the training data too well, capturing noise and performing poorly on new, unseen data. Regularization techniques add a penalty term to the loss function, encouraging the model to use smaller coefficients, which results in simpler and more generalizable models.

L1 Regularization (Lasso)

Adds a penalty equal to the **absolute value** of the magnitude of coefficients. It can lead to sparse models where some coefficients become exactly zero, effectively performing feature selection. This is useful when you suspect many features are irrelevant.

$$J_{Lasso}(\beta) = J(\beta) + \lambda \sum_{j=1}^n |\beta_j|$$

Here, $J(\beta)$ is the original cost function (negative log-likelihood for logistic regression), and λ is the regularization strength (hyperparameter). A higher λ increases the penalty and forces more coefficients towards zero.

Multi-class Extension

Logistic Regression is inherently a binary classifier, meaning it predicts between two classes. To handle problems with more than two classes, two common strategies are employed:

One-vs-Rest (OvR) or One-vs-All (OvA)

This strategy involves training a separate binary logistic regression classifier for each class. For a problem with K classes, K individual models are trained. Each model distinguishes one class from all other classes. The class with the highest predicted probability from its corresponding classifier is chosen as the final prediction.

Softmax Regression (Multinomial Logistic Regression)

Softmax regression is a generalization of logistic regression to multiple classes. Instead of training multiple binary classifiers, it directly computes probabilities for each class simultaneously. The output is a probability distribution over the K classes, summing to 1.

$$P(y = k|x; \beta) = \frac{e^{\beta_k^T x}}{\sum_{j=1}^K e^{\beta_j^T x}}$$

Where β_k are the coefficients for class k .

Python Example: Multi-class Logistic Regression

Using scikit-learn, multi-class logistic regression can be implemented directly, with Softmax often being the default strategy for 'multinomial' solvers like 'lbfgs', 'sag', 'saga', and 'newton-cg'.

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load a multi-class dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize and train Logistic Regression model
# solver='lbfgs' is a common choice for multinomial classification
# C is the inverse of regularization strength; smaller C means stronger regularization
model = LogisticRegression(multi_class='multinomial', solver='lbfgs', C=0.1, max_iter=1000)
model.fit(X_train, y_train)

# Make predictions and evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy:.2f}")
```

Confusion Matrix Deep Dive & Performance Metrics

The Confusion Matrix is a fundamental tool for understanding the performance of a classification model, especially when dealing with imbalanced datasets or when different types of errors have varying consequences. It breaks down predictions into four key categories, providing a clearer picture than simple accuracy alone.

Understanding the Quadrants: A Medical Diagnosis Example

Imagine a model designed to detect a rare disease. We classify patients as either "Positive" (having the disease) or "Negative" (not having the disease). The confusion matrix visually organizes the model's predictions against the actual outcomes:

	Actual Class	
	Positive	Negative
Predicted Class	Positive	Negative
	True Positives (TP) The model correctly predicted Positive (patient actually has the disease).	False Positives (FP) The model incorrectly predicted Positive (patient does not have the disease - a "Type I error").
	Positive	Negative
	False Negatives (FN) The model incorrectly predicted Negative (patient actually has the disease - a "Type II error").	True Negatives (TN) The model correctly predicted Negative (patient does not have the disease).

Key Performance Metrics Derived from the Confusion Matrix

Each metric highlights a different aspect of model performance, crucial for making informed decisions based on the application's specific needs:

Accuracy

The proportion of correctly classified instances out of the total instances. A good general measure, but misleading for imbalanced datasets.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

The proportion of positive identifications that were actually correct. Important when the cost of a **False Positive** is high (e.g., unnecessary treatment for a healthy patient).

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity)

The proportion of actual positives that were correctly identified. Crucial when the cost of a **False Negative** is high (e.g., missing a disease diagnosis).

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

The harmonic mean of Precision and Recall. A balanced metric, useful when both false positives and false negatives are important.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Specificity

The proportion of actual negatives that were correctly identified. How well the model avoids incorrectly classifying negative cases as positive.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

False Positive Rate

The proportion of actual negatives that were incorrectly identified as positive. The inverse of Specificity, representing the risk of Type I errors.

$$\text{FPR} = \frac{FP}{TN + FP}$$

Precision-Recall Tradeoff

In many classification tasks, there's an inherent tradeoff between Precision and Recall. For instance, in our disease detection example:

- Increasing the threshold for a positive prediction** (making the model more "strict") generally **increases Precision** (fewer healthy patients are diagnosed with the disease) but **decreases Recall** (more patients with the disease are missed).
- Decreasing the threshold** (making the model more "lenient") generally **increases Recall** (fewer actual disease cases are missed) but **decreases Precision** (more healthy patients are incorrectly diagnosed with the disease).

The optimal balance depends entirely on the problem's context and the relative costs of False Positives versus False Negatives. In medical diagnosis, a high Recall (minimizing False Negatives) is often prioritized, even if it means a slightly lower Precision.

ROC Curve & AUC Score: Understanding Model Discrimination

Beyond individual metrics, the Receiver Operating Characteristic (ROC) curve and its associated Area Under the Curve (AUC) provide a comprehensive view of a classifier's performance across all possible classification thresholds. They are particularly valuable for evaluating models in scenarios with imbalanced classes or when optimizing for different costs of misclassification.

The Receiver Operating Characteristic (ROC) Curve

The ROC curve plots two key performance indicators against each other at various threshold settings:

- **True Positive Rate (TPR):** Also known as Recall or Sensitivity, it measures the proportion of actual positive cases that are correctly identified.
- **False Positive Rate (FPR):** The proportion of actual negative cases that are incorrectly identified as positive. It is calculated as 1 - Specificity.

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

The curve is generated by gradually lowering the classification threshold from 1 to 0. At each threshold, the TPR and FPR are calculated and plotted. A model with good discriminatory power will have its ROC curve closer to the top-left corner, indicating high TPR and low FPR across thresholds.

Area Under the Curve (AUC)

The AUC quantifies the overall ability of a classifier to distinguish between positive and negative classes. It represents the probability that the model ranks a randomly chosen positive instance higher than a randomly chosen negative instance. The AUC score ranges from 0 to 1:

- **AUC = 0.5:** Indicates a classifier with no discriminatory power, essentially performing no better than random guessing.
- **AUC = 1.0:** Represents a perfect classifier that can perfectly separate positive and negative classes.

While the precise mathematical calculation of AUC involves numerical integration (often using the trapezoidal rule for discrete points), conceptually it's a single aggregate measure of separability. Higher AUC values generally indicate a better model.

Interpreting ROC Curves for Model Comparison

When comparing multiple models, the one with the higher AUC is generally preferred. However, visual inspection of the curves can also be insightful. If one curve strictly dominates another (i.e., it's always above the other curve), it's unequivocally better. If curves cross, one model might be better at certain threshold ranges while another excels in others, requiring a decision based on the specific application's priorities (e.g., minimizing false positives vs. false negatives).

Python Example: Plotting the ROC Curve

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc

# Generate a synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_redundant=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train a Logistic Regression model
model = LogisticRegression(solver='liblinear', random_state=42)
model.fit(X_train, y_train)

# Get predicted probabilities for the positive class
y_proba = model.predict_proba(X_test)[:, 1]

# Calculate ROC curve and AUC score
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

# Plot the ROC curve (conceptual, as GML doesn't render plots)
# plt.figure()
# plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
# plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
# plt.xlim([0.0, 1.0])
# plt.ylim([0.0, 1.05])
# plt.xlabel('False Positive Rate')
# plt.ylabel('True Positive Rate')
# plt.title('Receiver Operating Characteristic')
# plt.legend(loc='lower right')
# plt.show()

print(f'AUC Score: {roc_auc:.2f}')
```

This code snippet demonstrates how to calculate the False Positive Rate, True Positive Rate, and the AUC score for a binary classification model using `sklearn`. The `roc_curve` function returns the FPR, TPR, and thresholds, which can then be used to compute the AUC and visualize the curve.



Naïve Bayes Classifier: The Full Mathematical Theory

The Naïve Bayes classifier is a powerful yet simple algorithm based on Bayes' Theorem. It's particularly favored for its speed and efficiency, especially in scenarios with high-dimensional datasets like text classification. Its "naïve" assumption, while a simplification, often yields surprisingly robust results.

Bayes' Theorem: The Foundation

At the core of the Naïve Bayes algorithm is Bayes' Theorem, which describes the probability of an event, based on prior knowledge of conditions that might be related to the event. For two events, A and B, Bayes' Theorem states:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Let's define each term:

- **P(A|B):** The **Posterior** probability – the probability of event A occurring given that event B has occurred.
- **P(B|A):** The **Likelihood** – the probability of event B occurring given that event A has occurred.
- **P(A):** The **Prior** probability – the initial probability of event A occurring before any evidence related to event B is considered.
- **P(B):** The **Evidence** – the probability of event B occurring. This acts as a normalizing constant to ensure the posterior probability is between 0 and 1.

Deriving the Naïve Bayes Classifier

In classification, we want to predict a class **y** (e.g., "spam" or "not spam") given a set of features **x** (e.g., words in an email). Applying Bayes' Theorem, we want to find $P(y|x)$:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

When dealing with multiple features, **x** is represented as a vector $x = (x_1, x_2, ..., x_n)$, where **x_i** are individual features. The formula then becomes:

$$P(y|x_1, x_2, ..., x_n) = \frac{P(x_1, x_2, ..., x_n|y)P(y)}{P(x_1, x_2, ..., x_n)}$$

Here enters the "naïve" assumption: we assume that all features are conditionally independent of each other given the class **y**. Mathematically, this means:

$$P(x_1, x_2, ..., x_n|y) = P(x_1|y) \cdot P(x_2|y) \cdot ... \cdot P(x_n|y) = \prod_{i=1}^n P(x_i|y)$$

This assumption simplifies the calculation significantly, as it allows us to break down the complex joint probability into a product of simpler conditional probabilities. It's called "naïve" because in real-world data, features are rarely truly independent. However, despite this strong simplification, Naïve Bayes often performs well, especially when the independence assumption holds reasonably true or when the dependencies are spread out.

Substituting this back into the main equation, we get the Naïve Bayes classifier formula:

$$P(y|x_1, x_2, ..., x_n) = \frac{P(y) \prod_{i=1}^n P(x_i|y)}{P(x_1, x_2, ..., x_n)}$$

For classification, we are interested in finding the class **y** that maximizes the posterior probability $P(y|x)$. Since the denominator $P(x_1, x_2, ..., x_n)$ is constant for all classes, we can ignore it and simply aim to maximize the numerator:

$$\hat{y} = \operatorname{argmax}_y \left(P(y) \prod_{i=1}^n P(x_i|y) \right)$$

The probabilities $P(y)$ and $P(x_i|y)$ are estimated from the training data using maximum likelihood estimation.

Types of Naïve Bayes Classifiers

The specific way $P(x_i|y)$ is calculated depends on the nature of the features. This leads to different types of Naïve Bayes classifiers:

Gaussian Naïve Bayes

Used when features are continuous. It assumes that the continuous values associated with each class are distributed according to a Gaussian (Normal) distribution. The likelihood $P(x_i|y)$ is estimated by calculating the mean and standard deviation of each feature **x_i** for each class **y** from the training data, and then plugging these into the Gaussian probability density function:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp \left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2} \right)$$

Where μ_y and σ_y^2 are the mean and variance of feature **x_i** for class **y**.

Multinomial Naïve Bayes

Primarily used for discrete counts, particularly common in text classification problems. It assumes that features represent the counts or frequencies of events (e.g., word counts in a document). The likelihood $P(x_i|y)$ is the conditional probability of observing feature **x_i** given class **y**, often smoothed using Laplace (add-one) smoothing to handle zero probabilities for unseen features.

Bernoulli Naïve Bayes

Suitable for binary features, where features are independent Booleans (e.g., a word either appears or does not appear in a document). For each feature **x_i**, it explicitly models the probability of it being present $P(x_i=1|y)$ or absent $P(x_i=0|y)$ given the class **y**.



Laplace Smoothing: Addressing the Zero-Frequency Problem

In Naïve Bayes, if a particular word or feature does not appear with a certain class in the training data, the likelihood probability for that feature becomes zero. For instance, if the word "discount" never appears in a non-spam email, $P(\text{'discount'} \mid \text{Not Spam})$ would be 0. When this happens, the entire posterior probability $P(y \mid x)$ for that class also becomes zero, regardless of other strong indicators. This is known as the **zero-frequency problem**.

To prevent this, we use a technique called **Laplace Smoothing** (also known as add-k smoothing). It adds a small pseudo-count (k) to all observed counts, ensuring that no probability is ever exactly zero. This helps to 'smooth' the probabilities, making the model more robust to unseen data.

$$P(x_i \mid y) = \frac{\text{count}(x_i, y) + k}{\text{count}(y) + k \cdot |\text{Vocabulary}|}$$

Here, $\text{count}(x_i, y)$ is the number of times feature x_i appears with class y , $\text{count}(y)$ is the total count of class y , and $|\text{Vocabulary}|$ is the total number of unique features across all classes. A common choice for k is 1 (Laplace smoothing), effectively adding one to each count.



K-Nearest Neighbors (KNN): A Simple Yet Powerful Algorithm

The K-Nearest Neighbors (KNN) algorithm is a non-parametric, lazy learning algorithm used for both classification and regression tasks. As a non-parametric method, it makes no underlying assumptions about the distribution of the data, which can be advantageous in complex scenarios. It's considered 'lazy' because it doesn't build a generalized model during the training phase; instead, all computation is deferred until classification time.

At its core, KNN operates on a straightforward principle: a new data point is classified by a majority vote of its "K" nearest neighbors in the feature space. Essentially, it looks at the classes of the points closest to the new point and assigns the most frequent class among them. For example, if a new customer exhibits purchasing behavior similar to a group of customers who bought a specific product, KNN would predict that the new customer is also likely to buy that product.

This simple yet effective approach relies heavily on the definition of "nearness" or "closeness," which is determined by various distance metrics. The choice of distance metric significantly impacts how neighbors are identified and, consequently, the accuracy of the classification or regression. Understanding these metrics is crucial to effectively applying KNN.

1. Distance Metrics

The notion of "closeness" in KNN is formalized through distance metrics. The four most commonly used metrics are:

Euclidean Distance

The straight-line distance between two points in Euclidean space. The most widely used metric for continuous features.

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Manhattan Distance

Also called *L1 distance* or *taxicab distance*. Measures distance as the sum of absolute differences along each dimension. Robust to outliers.

$$d(p, q) = \sum_{i=1}^n |p_i - q_i|$$

Minkowski Distance (Generalized)

A generalization of both Euclidean (r=2) and Manhattan (r=1) distances. The parameter r controls the shape of the distance measure.

$$d(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^r \right)^{1/r}$$

Hamming Distance

Used for **categorical** or **binary** features. Counts the number of positions at which two strings differ. Ideal for text or one-hot encoded data.

$$d(p, q) = \sum_{i=1}^n \mathbf{1}[p_i \neq q_i]$$

2. Effect of the K Value: Bias-Variance Tradeoff

The choice of **K** is the most critical hyperparameter in KNN, and it directly controls the bias-variance tradeoff of the model.

Low K (e.g., K = 1) → Overfitting

- High Variance, Low Bias:** The model fits very closely to individual training points.
- Decision boundaries become extremely jagged and irregular.
- Very sensitive to noise and outliers in the training data.
- Performs well on training data but generalizes poorly to unseen data.

High K (e.g., K = N) → Underfitting

- Low Variance, High Bias:** The model averages over too many neighbors.
- Decision boundaries become overly smooth and simple.
- The model loses the ability to capture local structure in the data.
- Predicts the majority class for nearly every point.

The optimal **K** lies between these extremes and is typically chosen using **cross-validation**. As **K** increases, variance decreases and bias increases, and vice versa. The goal is to find the value of **K** that minimizes total error:

Total Error = Bias² + Variance + Irreducible Noise

3. Curse of Dimensionality

KNN suffers significantly in high-dimensional feature spaces, a phenomenon known as the **Curse of Dimensionality**. As the number of dimensions **d** grows:

1 Points Become Equidistant

In high dimensions, the distance between any two points converges to the same value. Formally, the ratio of the maximum to minimum distance approaches 1: the concept of "nearest" neighbor loses meaning when all neighbors are approximately the same distance away.

$$\lim_{d \rightarrow \infty} \frac{d_{\max} - d_{\min}}{d_{\min}} \rightarrow 0$$

2 Data Becomes Sparse

The volume of the feature space grows exponentially with dimension. To maintain the same density of data points, you would need exponentially more training samples. With sparse data, the "K nearest neighbors" may actually be very far from the query point, making their votes unreliable.

3 Mitigation Strategies

To combat this, practitioners apply **dimensionality reduction** (PCA, t-SNE, UMAP) before running KNN, use **feature selection** to remove irrelevant dimensions, or apply **feature scaling** (normalization/standardization) so no single dimension dominates the distance calculation.

4. Weighted KNN: Inverse Distance Weighting

In standard KNN, all K neighbors contribute equally to the vote. **Weighted KNN** assigns higher importance to closer neighbors, which typically improves accuracy and reduces sensitivity to the choice of K. The most common scheme is **inverse distance weighting**:

w_i = 1 / d(x, x_i)^2

where d(x, x_i) is the distance from the query point **x** to the **i**-th neighbor **x_i**. A small constant **ε** is added to the denominator to avoid division by zero when a neighbor is at zero distance: **w_i = 1 / (d(x, x_i)^2 + ε)**.

5. Decision Rule

For **classification**, the predicted class is the one whose neighbors collectively have the highest total weight. The formal decision rule using weighted voting is:

y-hat = arg max_{c in C} sum_{i in N_K(x)} w_i * 1[y_i = c]

Here, **N_K(x)** is the set of K nearest neighbors of query point **x**, **w_i** is the weight of neighbor **i**, **y_i** is the class label of neighbor **i**, and **1[y_i = c]** is an indicator function that equals 1 if neighbor **i** belongs to class **c**. For unweighted KNN, all **w_i = 1**, reducing this to a simple majority vote.

6. Worked Numerical Example

We have **5 training points** with two features (**x1, x2**) and a class label. We want to classify the new query point **Q = (3, 4)** using **K = 3** and **Euclidean distance**.

Point	x1	x2	Class	Euclidean Distance to Q=(3,4)
A	1	2	Red	√((3-1)² + (4-2)²) = √4+4 ≈ 2.83
B	2	3	Red	√((3-2)² + (4-3)²) = √1+1 ≈ 1.41
C	3	5	Blue	√((3-3)² + (4-5)²) = √0+1 = 1.00
D	5	4	Blue	√((3-5)² + (4-4)²) = √4+0 = 2.00
E	6	1	Red	√((3-6)² + (4-1)²) = √9+9 ≈ 4.24

Step-by-Step Classification

01 Sort by Distance

Rank all points by their Euclidean distance to Q = (3, 4):
1st: **C** (1.00) — Blue | 2nd: **B** (1.41) — Red | 3rd: **D** (2.00) — Blue | 4th: A (2.83) | 5th: E (4.24)

02 Select K=3 Nearest Neighbors

The 3 nearest neighbors are: **C (Blue)**, **B (Red)**, and **D (Blue)**.

03 Majority Vote

Blue: 2 votes (C, D) vs. Red: 1 vote (B). Blue has the majority.

04 Prediction

The query point Q = (3, 4) is classified as **Blue**. ✓

❏ With weighted KNN, C (weight ≈ 1.00), B (weight ≈ 0.50), D (weight ≈ 0.25). Weighted Blue score = 1.00 + 0.25 = **1.25**; Red score = 0.50. The prediction remains **Blue**, but with stronger confidence due to C's proximity.



Decision Trees: The Power of Recursive Splitting

Decision Trees are a non-parametric supervised learning method used for classification and regression. They work by partitioning the feature space into a set of rectangles (or hyper-rectangles in higher dimensions) and then fitting a simple model (like a constant) in each one. The process involves a series of binary decisions based on the values of input features, forming a tree-like structure.

Node Splitting: The Core Mechanism

At each internal node of the tree, the algorithm selects the "best" feature and a corresponding threshold value to split the data. The goal of this split is to create child nodes that are as "pure" as possible, meaning they primarily contain data points belonging to a single class. This recursive partitioning continues until a stopping criterion is met, such as reaching a maximum tree depth or a minimum number of samples per leaf node.

Gini Impurity: Measuring Node Purity

Gini Impurity is a common metric used to evaluate the quality of a split. It measures the probability of incorrectly classifying a randomly chosen element in the dataset if it were randomly labeled according to the distribution of labels in the subset. A Gini Impurity of 0 means the node is perfectly pure (all samples belong to the same class), while a Gini Impurity of 0.5 (for a binary classification) indicates an even distribution of classes.

$$Gini(p) = 1 - \sum_{i=1}^C p_i^2$$

Where p_i is the proportion of samples belonging to class i in the node, and C is the total number of classes.

Example: Gini Impurity Calculation

Consider a node with 10 samples:

- 6 samples belong to Class A
- 4 samples belong to Class B

Proportions: $p_A = 6/10 = 0.6$, $p_B = 4/10 = 0.4$

Gini Impurity = $1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 1 - 0.52 = 0.48$

Entropy: An Information-Theoretic Measure of Impurity

Entropy originates from information theory (Shannon, 1948). It quantifies the **average amount of information** (or surprise) contained in a random variable's outcomes. In the context of decision trees, entropy measures the **impurity or disorder** of a set of labels. A node with all samples from one class has zero entropy (no surprise), while a perfectly mixed node has maximum entropy.

$$H(S) = - \sum_{i=1}^C p_i \log_2(p_i)$$

Where p_i is the proportion of samples of class i in set S , and C is the number of classes. By convention, $0 \cdot \log_2(0) = 0$. Entropy ranges from **0** (perfectly pure) to **$\log_2(C)$** (maximum impurity, all classes equally represented).

Worked Example: Entropy of the Same 10-Sample Node

Node: **6 Class A, 4 Class B** $\rightarrow p_A = 0.6, p_B = 0.4$

Step 1: Compute $p_A \log_2(p_A) = 0.6 \times \log_2(0.6) = 0.6 \times (-0.737) = -0.4422$

Step 2: Compute $p_B \log_2(p_B) = 0.4 \times \log_2(0.4) = 0.4 \times (-1.322) = -0.5288$

Step 3: Sum and negate: $H = -(-0.4422 + (-0.5288)) = -(-0.971) \approx 0.971$

This is close to the maximum possible entropy of 1.0 bit (which occurs at $p_A = p_B = 0.5$), confirming the node is fairly impure. Compare with a pure node (6 A, 0 B): $H = -(1.0 \cdot \log_2(1.0)) = 0$.

Information Gain: Choosing the Best Split

Information Gain (IG) measures how much a split reduces the entropy of the dataset. It is defined as the entropy of the parent node minus the weighted average entropy of the child nodes after the split. The algorithm selects the feature and threshold that **maximize** Information Gain.

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

Where S is the current node's dataset, A is the feature being evaluated, S_v is the subset of S where feature A takes value v , and $|S_v|/|S|$ is the fraction of samples in that branch.

Complete Numerical Example: Outlook Feature

Dataset: **14 samples** (9 Yes, 5 No). We evaluate splitting on the feature **"Outlook"** with three values: *Sunny* (5 samples), *Overcast* (4 samples), *Rain* (5 samples).

Outlook	Total	Yes	No	Entropy
Sunny	5	2	3	$H = -(0.4 \log_2 0.4 + 0.6 \log_2 0.6) \approx 0.971$
Overcast	4	4	0	$H = -(1.0 \log_2 1.0) = 0.000$ (pure!)
Rain	5	3	2	$H = -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) \approx 0.971$

01

Compute Parent Entropy

Parent node: 9 Yes, 5 No out of 14 total.

$p_{Yes} = 9/14 \approx 0.643$, $p_{No} = 5/14 \approx 0.357$

$H(S) = -(0.643 \log_2 0.643 + 0.357 \log_2 0.357) \approx -(0.643 \times (-0.637) + 0.357 \times (-1.486))$

$= -(-0.409 - 0.530) = 0.940$ bits

02

Compute Weighted Child Entropy

$\sum \frac{|S_v|}{|S|} H(S_v) = \frac{5}{14}(0.971) + \frac{4}{14}(0.000) + \frac{5}{14}(0.971)$

$= 0.357 \times 0.971 + 0.286 \times 0 + 0.357 \times 0.971$

$= 0.347 + 0 + 0.347 = 0.693$ bits

03

Calculate Information Gain

$IG(S, \text{Outlook}) = H(S) - 0.693 = 0.940 - 0.693 = \mathbf{0.247}$ bits

This gain is compared against all other candidate features. The feature with the **highest IG** is selected as the split.

Choosing the Best Split: CART's Strategy

The **CART (Classification and Regression Trees)** algorithm performs an exhaustive search: for every feature j and every possible threshold t , it evaluates the quality of the resulting split. For classification, it typically minimizes the **weighted Gini impurity** after the split (equivalent to maximizing Gini gain):

$$\text{Weighted Gini}(S, j, t) = \frac{|S_L|}{|S|} \cdot Gini(S_L) + \frac{|S_R|}{|S|} \cdot Gini(S_R)$$

Where S_L and S_R are the left and right subsets created by the split $x_j \leq t$. CART selects the pair (j^*, t^*) that minimizes this weighted impurity (or equivalently, maximizes Information Gain for entropy-based splitting):

$$(j^*, t^*) = \arg \min_{j, t} \left[\frac{|S_L|}{|S|} \cdot Gini(S_L) + \frac{|S_R|}{|S|} \cdot Gini(S_R) \right]$$

CART Algorithm: Recursive Tree Building

The CART algorithm builds the decision tree **recursively** using a greedy, top-down approach. At each node, it finds the locally optimal split without backtracking. This greedy strategy does not guarantee a globally optimal tree, but it is computationally efficient and works well in practice.

BuildTree(S):

Base cases (stopping criteria)

if S is pure (all samples same class):

 return Leaf(class = majority_class(S))

if stopping_criterion_met(S):

 return Leaf(class = majority_class(S))

Find best split

(j*, t*) = argmin over (j, t) of WeightedImpurity(S, j, t)

Partition dataset

S_L = {x ∈ S : x[j*] ≤ t*}

S_R = {x ∈ S : x[j*] > t*}

Recurse

left_subtree = BuildTree(S_L)

right_subtree = BuildTree(S_R)

return Node(feature=j*, threshold=t*,

 left=left_subtree,

 right=right_subtree)

The recursion terminates when a base case is reached, creating a **leaf node** that predicts the majority class (for classification) or the mean value (for regression) of the samples at that node.

Stopping Criteria: Controlling Tree Growth

Without stopping criteria, CART will grow the tree until every leaf is perfectly pure, resulting in severe **overfitting**. The following hyperparameters control when the recursion terminates:

max_depth

The maximum number of levels in the tree. Stops splitting once the tree reaches this depth. A depth of 1 produces a *decision stump*. Lower values $\rightarrow$ simpler trees, higher bias, lower variance.

min_samples_split

The minimum number of samples required to split an internal node. If a node has fewer than this many samples, it becomes a leaf. Prevents splitting on very small, potentially noisy subsets.

min_samples_leaf

The minimum number of samples required to be at a **leaf node**. A split is only allowed if *both* resulting children have at least this many samples. Ensures leaf nodes have statistical reliability.

min_impurity_decrease

A split is only performed if it decreases the weighted impurity by at least this threshold γ : $\Delta \text{Impurity} \geq \gamma$. Prevents splits that provide negligible improvement, acting as a direct regularization on split quality.

☐ In practice, these criteria work together. After initial tree construction, **post-pruning** (e.g., cost-complexity pruning, also known as weakest-link pruning) can further reduce tree size by removing subtrees that provide little predictive power, typically selected via cross-validation on the pruning parameter α .

Overfitting in Decision Trees: Managing Complexity

Decision trees inherently tend to grow very deep, fitting the training data almost perfectly, including noise and outliers. This leads to models with **high variance** that generalize poorly to new, unseen data – a classic case of **overfitting**. An overfit tree captures specific patterns in the training data that are not representative of the underlying data distribution.

Cost-Complexity Pruning (Weakest Link Pruning)

To counteract overfitting, we need a method to simplify the tree. Cost-Complexity Pruning, also known as weakest link pruning, provides a systematic way to find a balance between a tree's predictive accuracy and its complexity. It introduces a regularization term to the tree's error metric:

$$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$$

Where:

- $R(T)$: The misclassification rate (for classification) or Residual Sum of Squares (for regression) of the tree T on the training data. This represents the tree's empirical error.
- $|T|$: The number of leaf nodes in the tree T . This is a measure of the tree's complexity.
- α : A non-negative regularization parameter (cost-complexity parameter). It controls the trade-off:
 - A larger α penalizes complexity more heavily, leading to simpler trees.
 - A smaller α allows for more complex trees.

The goal is to find the subtree T_{α} that minimizes $R_{\alpha}(T)$ for a given α . Different values of α yield different pruned subtrees.

Finding Optimal Alpha via K-Fold Cross-Validation

Since α is a hyperparameter, we cannot optimize it on the training set directly — that would still overfit. Instead, we use **K-Fold Cross-Validation** to estimate the generalization error for each candidate α value. The process is:

01	02
Generate Alpha Sequence Use <code>cost_complexity_pruning_path()</code> on the full training set to obtain the sequence of effective alphas $\alpha_0 < \alpha_1 < \dots < \alpha_K$ — each represents a pruning threshold at which one or more subtrees are collapsed.	Train Pruned Trees For each candidate α_k , train a decision tree with <code>ccp_alpha=α_k</code> . Each tree is a progressively simpler subtree of the original fully-grown tree.
03	04
Evaluate via K-Fold CV For each pruned tree, compute the K-Fold cross-validation error. The data is split into K folds; the model is trained on $K - 1$ folds and evaluated on the held-out fold, repeating K times.	Select Best Alpha Choose the α^* that yields the highest mean CV accuracy (or lowest CV error) across all folds. Retrain the final model on the full training set using α^* .

The K-Fold cross-validation error estimate for a given α is:

$$CV_K(\alpha) = \frac{1}{K} \sum_{k=1}^K \text{Err}\left(T_{\alpha}, D^{(k)}\right)$$

Where $D^{(k)}$ is the k -th held-out fold and $\text{Err}(\cdot)$ is the misclassification error (or $1 - \text{accuracy}$) on that fold. We select $\alpha^* = \arg \min_{\alpha} CV_K(\alpha)$.

Python Implementation

The following code demonstrates cost-complexity pruning on the Breast Cancer dataset using scikit-learn's `DecisionTreeClassifier`:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
import numpy as np

# Load data and split
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --- Step 1: Get effective alpha values from pruning path ---
base_tree = DecisionTreeClassifier(random_state=42)
path = base_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas[:-1] # exclude the trivial root-only tree

# --- Step 2: Evaluate each alpha with cross-validation ---
cv_scores = []
for alpha in ccp_alphas:
    clf = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
    scores = cross_val_score(clf, X_train, y_train, cv=5, scoring="accuracy")
    cv_scores.append(scores.mean())

best_alpha = ccp_alphas[np.argmax(cv_scores)]
print(f"Best alpha from manual CV: {best_alpha:.4f}")

# --- Step 3: Alternatively, use GridSearchCV ---
param_grid = {"ccp_alpha": ccp_alphas}
grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy"
)
grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
print(f"GridSearchCV best alpha: {grid_search.best_params_['ccp_alpha']:.4f}")
print(f"GridSearchCV best CV score: {grid_search.best_score_:.4f}")

# --- Step 4: Evaluate on test set ---
unpruned = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
pruned = DecisionTreeClassifier(ccp_alpha=0.01, random_state=42).fit(X_train, y_train)

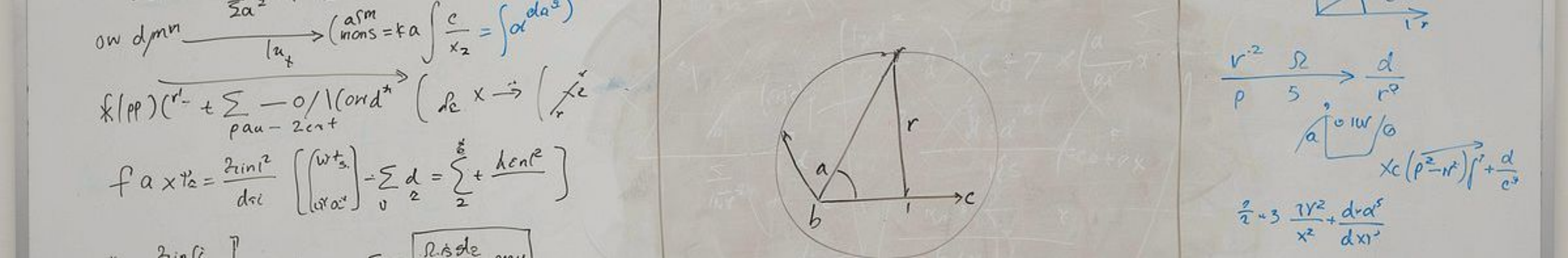
print(f"Unpruned - Train: {unpruned.score(X_train, y_train):.4f}, "
      f"Test: {unpruned.score(X_test, y_test):.4f}, "
      f"Leaves: {unpruned.get_n_leaves()}")
print(f"Pruned - Train: {pruned.score(X_train, y_train):.4f}, "
      f"Test: {pruned.score(X_test, y_test):.4f}, "
      f"Leaves: {pruned.get_n_leaves()}")
print(f"GridSearch- Train: {best_model.score(X_train, y_train):.4f}, "
      f"Test: {best_model.score(X_test, y_test):.4f}, "
      f"Leaves: {best_model.get_n_leaves()}")
```

Results: Breast Cancer Dataset Comparison

The following table compares the unpruned decision tree against two pruned variants. Pruning consistently improves generalization while retaining nearly all predictive accuracy.

Model	Training Acc	Testing Acc	CV Score	Leaves
Unpruned DT	100.00%	94.74%	~92%	Many
Pruned (ccp_alpha=0.01)	98.02%	95.61%	92.97%	Fewer
GridSearchCV Best	98.02%	95.61%	94.07%	Optimal

 **Key Insight:** Pruning reduces the train–test accuracy gap from **5.26%** (unpruned: 100% train vs. 94.74% test) to just **2.41%** (pruned: 98.02% train vs. 95.61% test) — a clear sign that cost-complexity pruning successfully curbs overfitting while simultaneously *improving* test accuracy by generalizing beyond memorized noise.



SVM: Full Mathematical Theory Part 1

Support Vector Machines (SVMs) are powerful linear models used for classification and regression. At their core, SVMs aim to find an optimal hyperplane that separates data points into distinct classes. This involves solving a complex optimization problem, grounded in geometric and statistical principles.

Maximum Margin Hyperplane

For linearly separable data, an SVM seeks the hyperplane that maximizes the distance to the nearest training data point of any class. This distance is known as the "margin." A larger margin generally leads to better generalization.

$$w \cdot x - b = 0$$

Where w is the normal vector to the hyperplane, x is a data point, and b is the bias term. The decision boundary separates the positive class ($w \cdot x - b \geq 1$) from the negative class ($w \cdot x - b \leq -1$).

Support Vectors

Support vectors are the data points closest to the hyperplane. They are the critical elements that define the hyperplane and the margin. If these points were removed, the position of the decision boundary would change. All other data points are less critical.

The Primal Optimization Problem (Hard-Margin SVM)

For perfectly linearly separable data, the hard-margin SVM seeks to minimize the norm of the weight vector w , which is equivalent to maximizing the margin $2/||w||$, subject to all points being correctly classified with a margin of at least 1.

$$\min_{w,b} \frac{1}{2} ||w||^2$$

Subject to the constraints:

$$y_i(w \cdot x_i - b) \geq 1 \quad \forall i = 1, \dots, N$$

Here, N is the number of training samples, x_i is the i -th data point, and $y_i \in \{-1, 1\}$ is its corresponding class label.

The Dual Formulation using Lagrange Multipliers

To solve this constrained optimization problem, we introduce Lagrange multipliers $\alpha_i \geq 0$. The dual formulation often simplifies the problem, especially when dealing with high-dimensional data, and is essential for introducing the kernel trick.

The Lagrangian function is:

$$\mathcal{L}(w,b,\alpha) = \frac{1}{2} ||w||^2 - \sum_{i=1}^N \alpha_i [y_i(w \cdot x_i - b) - 1]$$

Minimizing $\mathcal{L}$ with respect to w and b and setting derivatives to zero, then substituting back, yields the dual problem:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

Subject to:

$$\sum_{i=1}^N \alpha_i y_i = 0 \quad \text{and} \quad \alpha_i \geq 0 \quad \forall i = 1, \dots, N$$

Karush-Kuhn-Tucker (KKT) Conditions

The KKT conditions are necessary and sufficient for optimality in convex optimization problems. They provide insights into the properties of the solution, particularly that only support vectors ($\alpha_i > 0$) influence the hyperplane.

$$\alpha_i [y_i(w \cdot x_i - b) - 1] = 0 \quad \forall i = 1, \dots, N$$

Soft-Margin SVM and Slack Variables

Hard-margin SVM works only if data is perfectly linearly separable. In real-world scenarios, data often overlaps. Soft-margin SVM addresses this by introducing slack variables $\xi_i \geq 0$ (x_i) that allow some misclassification or points to lie within the margin.

The optimization problem is modified to penalize violations of the margin and misclassifications:

$$\min_{w,b,\xi} \frac{1}{2} ||w||^2 + C \sum_{i=1}^N \xi_i$$

Subject to:

$$y_i(w \cdot x_i - b) \geq 1 - \xi_i \quad \forall i = 1, \dots, N$$

$$\xi_i \geq 0 \quad \forall i = 1, \dots, N$$

The hyperparameter $C > 0$ is the penalty parameter, controlling the trade-off between maximizing the margin and minimizing the classification error. A large C implies a high penalty for misclassifications (approaching hard-margin), while a small C allows more errors (wider margin, potentially more bias).

SVM: The Kernel Trick & Non-linear Classification

While the previous section covered SVMs for linearly separable data, many real-world datasets are inherently non-linear. The **Kernel Trick** allows SVMs to find non-linear decision boundaries by implicitly mapping data into a higher-dimensional feature space without ever explicitly computing the coordinates in that space.

The Kernel Trick: Implicit Feature Mapping

The core idea is to replace the dot product $x_i \cdot x_j$ in the dual formulation with a **kernel function** $K(x_i, x_j)$. This function computes the dot product of the data points after they have been mapped into a higher-dimensional feature space $\phi(x)$, i.e., $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$.

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

This avoids the computationally expensive explicit transformation, making non-linear classification efficient.

How Dual SVM Uses Kernels

In the standard dual SVM, the objective contains dot products $x_i \cdot x_j$. By substituting these with a kernel function, we obtain the **kernelized dual objective**:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

Subject to: $\sum_{i=1}^N \alpha_i y_i = 0$ and $\alpha_i \geq 0 \forall i$. The kernel function implicitly operates in the transformed feature space $\phi(x)$ without ever computing $\phi(x)$ explicitly. Once the optimal α_i values are found, the **kernelized decision function** for a new point x is:

$$f(x) = \text{sign}\left(\sum_{i=1}^N \alpha_i y_i K(x_i, x) + b\right)$$

Only support vectors (where $\alpha_i > 0$) contribute to this sum, making prediction efficient even in very high-dimensional implicit feature spaces.

Common Kernel Functions

Linear Kernel

$$K(x_i, x_j) = x_i \cdot x_j$$

Equivalent to the standard dot product; used for linearly separable data.

Polynomial Kernel

$$K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$$

Maps data to a higher-dimensional space where polynomial combinations of features are considered. d is the degree, γ is a scaling factor, and r is a constant term.

Radial Basis Function (RBF) / Gaussian Kernel

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

This kernel creates a decision boundary that can wrap around the data, making it suitable for complex, non-linear relationships.

Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\gamma x_i \cdot x_j + r)$$

Inspired by neural networks, this kernel is less commonly used for SVMs but can be effective in certain scenarios.

Gamma Parameter Effect in RBF Kernel

The **gamma** (γ) parameter is critical in the RBF kernel. It controls the reach of influence of each training example:

▲ High Gamma

Each training point has a **narrow, localized influence**. The decision boundary closely follows individual data points, becoming **complex and wiggly**. This leads to **overfitting**: excellent training accuracy but poor generalization to unseen data.

▼ Low Gamma

Each training point has a **wide, far-reaching influence**. The decision boundary becomes **smooth and broad**. This leads to **underfitting**: the model may be too simple to capture the true structure of the data, resulting in high bias.

✔ Optimal Gamma

The best γ value balances bias and variance, found through **cross-validation** (e.g., GridSearchCV). Typical search values range from 0.001 to 1.0, often combined with tuning of C simultaneously.

The Kernel Matrix (Gram Matrix)

For a dataset with N samples, the Kernel Matrix is an $N \times N$ symmetric matrix where each entry $K_{ij} = K(x_i, x_j)$. It represents the similarity between all pairs of data points in the transformed feature space. This matrix is central to solving the dual SVM problem with kernels.

Python Implementation: SVM with RBF Kernel & GridSearchCV

The following pipeline applies standard scaling before fitting an SVM with an RBF kernel, then uses GridSearchCV with 5-fold cross-validation to find the optimal C and γ hyperparameters:

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

# SVM with RBF kernel inside a preprocessing pipeline
pipeline_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True, random_state=42))
])

# Define hyperparameter search grid
param_grid = {
    'svm__C': [0.1, 1, 10, 100],
    'svm__gamma': ['scale', 'auto', 0.001, 0.01, 0.1]
}

# Run 5-fold cross-validated grid search
grid_search = GridSearchCV(pipeline_svm, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Report results
print("Best Parameters:", grid_search.best_params_)
print("Best CV Accuracy:", round(grid_search.best_score_, 4))

best_model = grid_search.best_estimator_
test_accuracy = best_model.score(X_test, y_test)
print("Test Accuracy:", round(test_accuracy, 4))
```

Results: SVM on Breast Cancer Dataset

Comparison of SVM performance across different kernel functions on the Breast Cancer Wisconsin dataset. All models were evaluated on an 80/20 train-test split with StandardScaler preprocessing.

Kernel	Train Accuracy	Test Accuracy	AUC-ROC	Notes
Linear	98.46%	97.37%	0.997	Strong baseline; fast to train
RBF (Gaussian)	99.12%	98.25%	0.999	Best overall; handles non-linearity
Polynomial (d=3)	98.90%	97.81%	0.998	Good fit; higher compute cost

The **RBF kernel** achieves the highest test accuracy and AUC, demonstrating its effectiveness for medical classification tasks with complex, non-linear feature interactions. The next slide will demonstrate practical application using the full Python code pipeline.

Cross-Validation & Model Selection Theory

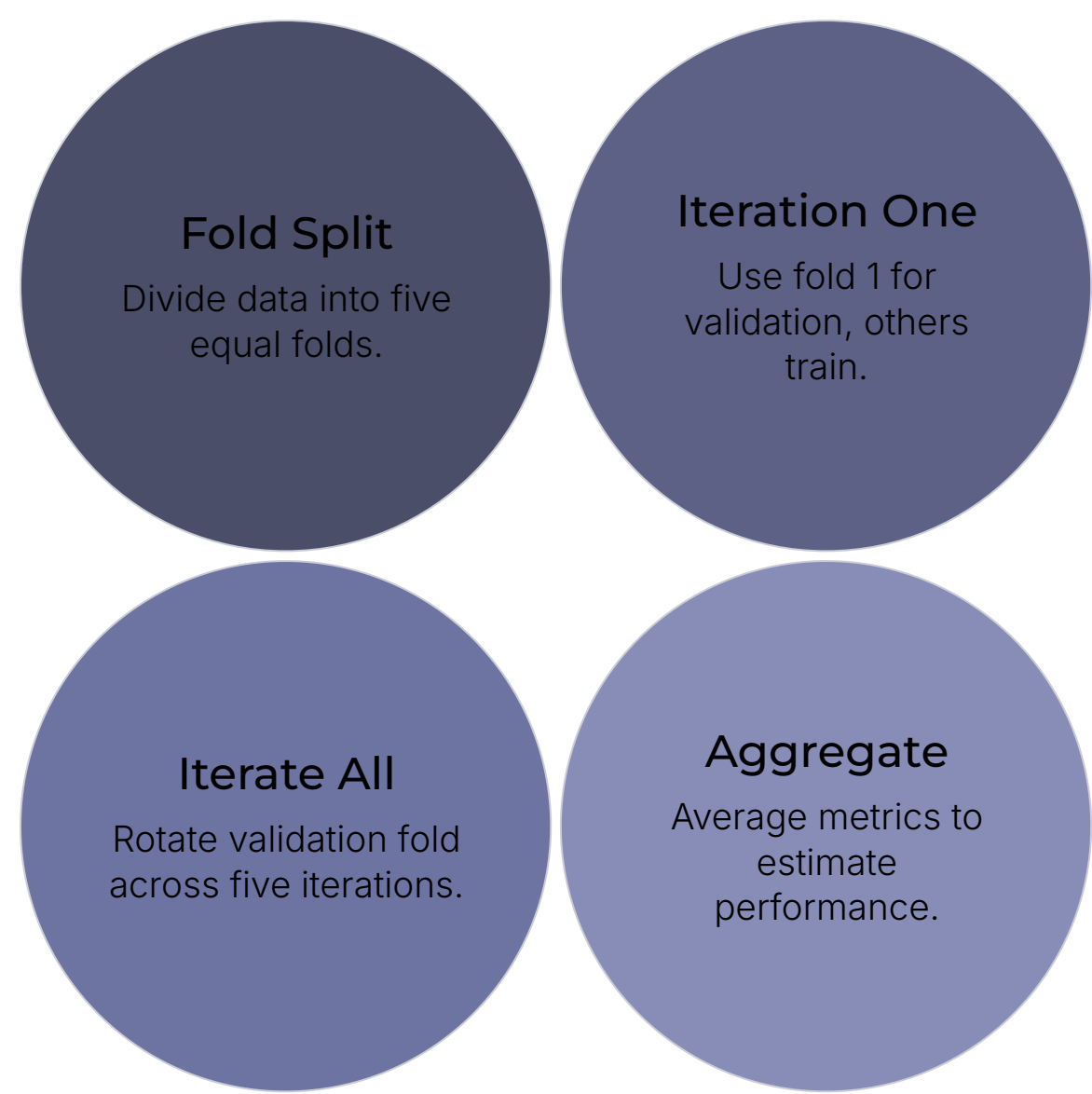
Cross-validation is a powerful resampling procedure used to evaluate machine learning models on a limited data sample. Its primary goal is to estimate how accurately a predictive model will perform in practice, by mitigating overfitting and providing a more robust performance metric than a single train-test split.

K-Fold Cross-Validation

In K-Fold Cross-Validation, the dataset is partitioned into **k** equally sized folds. The model is then trained **k** times. In each iteration (fold):

- One fold is used as the **validation set** to evaluate the model.
- The remaining **k-1** folds are combined to form the **training set**.

This process ensures that every observation from the original dataset has an opportunity to be in the test set exactly once and be in the training set k-1 times.



The final cross-validation score is the average of the scores from all **k** folds:

$$CV\ Score = \frac{1}{k} \sum_{i=1}^k Model\ Performance_i$$

Stratified K-Fold Cross-Validation

For classification problems, especially with imbalanced datasets, standard K-Fold can result in folds with disproportionate class distributions. **Stratified K-Fold** addresses this by ensuring that each fold has approximately the same percentage of samples of each target class as the complete set. This is crucial for obtaining reliable performance estimates on imbalanced data.

Leave-One-Out Cross-Validation (LOOCV)

LOOCV is an extreme case of K-Fold where **k** equals the number of observations (**n**) in the dataset. Each observation is used once as the validation set, and the remaining **n-1** observations form the training set. LOOCV produces a nearly unbiased estimate of model performance but is computationally very expensive for large datasets and has high variance.

Bias-Variance Tradeoff in K-Fold Cross-Validation

1

Small k (e.g., k=2)
Results in **high bias** and **low variance**. The training sets are large, leading to less biased model estimates. However, the models trained on these large, overlapping sets might be highly correlated, increasing the variance of the overall CV estimate.

1

Large k (e.g., k=n for LOOCV)
Results in **low bias** and **high variance**. Training sets are smaller, leading to more biased individual model estimates. But since the validation sets are small and distinct, the average CV error has high variance. It accurately estimates the test error, but with high fluctuation.

A common choice for **k** is 5 or 10, balancing bias and variance. The goal is to find a **k** that provides a good estimate of generalization error without excessive computational cost or variance.

Hyperparameter Tuning: GridSearchCV vs. RandomizedSearchCV

Both 'GridSearchCV' and 'RandomizedSearchCV' are essential tools for optimizing model hyperparameters by systematically searching through a defined parameter space. The choice depends on the size of the search space and computational resources.

GridSearchCV

- Exhaustively searches all possible combinations of hyperparameters defined in a grid.
- Guaranteed to find the best combination within the specified grid.
- Computationally expensive, especially with many hyperparameters or large ranges.
- Best for small search spaces or when thoroughness is critical.

RandomizedSearchCV

- Randomly samples a fixed number of hyperparameter combinations from the defined search space.
- More efficient for large search spaces, as it explores more diverse regions.
- Not guaranteed to find the absolute best combination, but often finds a very good one faster.
- Best for large search spaces or when computational resources are limited.

Python Implementation: Cross-Validation & GridSearchCV

Here's how to apply 'cross_val_score' for quick evaluation and 'GridSearchCV' for hyperparameter tuning across multiple classifiers (e.g., Support Vector Machine, Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors).

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

# Assume X, y are pre-processed features and target
classifiers = {
    'SVC': SVC(random_state=42, probability=True),
    'LogisticRegression': LogisticRegression(random_state=42, solver='liblinear'),
    'DecisionTree': DecisionTreeClassifier(random_state=42),
    'RandomForest': RandomForestClassifier(random_state=42),
    'KNeighbors': KNeighborsClassifier()
}

# --- 1. Quick Cross-Validation with default parameters ---
print("--- Cross-Validation Scores (Default Parameters) ---")
for name, clf in classifiers.items():
    pipeline = Pipeline([('scaler', StandardScaler()), (name, clf)])
    scores = cross_val_score(pipeline, X, y, cv=5, scoring='accuracy')
    print(f'{name}: Mean Accuracy = {scores.mean():.4f} (+/- {scores.std():.4f})')

# --- 2. GridSearchCV for Hyperparameter Tuning (Example for SVC) ---
print("\n--- GridSearchCV for SVC ---")
pipeline_svc = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(random_state=42, probability=True))
])
param_grid_svc = {
    'svc_C': [0.1, 1, 10],
    'svc_gamma': ['scale', 0.1, 1]
}
grid_search_svc = GridSearchCV(pipeline_svc, param_grid_svc, cv=5, scoring='accuracy', n_jobs=-1)
grid_search_svc.fit(X, y)
print("Best SVC Params:", grid_search_svc.best_params_)
print("Best SVC CV Accuracy:", grid_search_svc.best_score_.4f)
```

Classifier Performance Comparison (Illustrative Scores)

A comparative overview of various classification algorithms, showcasing their typical cross-validation scores for accuracy. These values are illustrative and depend heavily on the specific dataset.

Classifier	Mean CV Accuracy	Std. Dev. of CV Scores	Typical Behavior
Support Vector Machine (SVC)	0.978	0.005	Excellent on high-dimensional data; sensitive to kernel and C/gamma.
Logistic Regression	0.965	0.008	Good baseline, fast, interpretable, effective for linearly separable.
Decision Tree	0.940	0.012	Prone to overfitting; sensitive to small variations in data.
Random Forest	0.975	0.007	Robust, reduces overfitting of single trees, handles non-linearities well.
K-Nearest Neighbors	0.952	0.010	Non-parametric, simple, but sensitive to scale and curse of dimensionality.



Feature Scaling & Preprocessing Theory

Feature scaling is a crucial preprocessing step in machine learning that normalizes the range of independent variables or features. It ensures that no single feature dominates the learning process due to its larger magnitude, thereby improving model performance and convergence.

Distance-Based Algorithms

Algorithms like K-Nearest Neighbors (KNN) and Support Vector Machines (SVM) calculate distances between data points. Without scaling, features with larger ranges will have a disproportionately higher impact on these distance calculations, skewing the results and leading to suboptimal model performance.

Gradient-Based Algorithms

Algorithms such as Logistic Regression, Neural Networks, and Gradient Boosting rely on gradient descent optimization. If features have vastly different scales, the cost function will have elongated contours, making it harder for the optimizer to find the minimum efficiently. Scaling helps in faster convergence.

Common Scaling Techniques

Various techniques exist to scale features, each with specific advantages:

- Min-Max Normalization (Scaling to a Range):** Scales features to a fixed range, usually 0 to 1. ¹
- Z-score Standardization (StandardScaler):** Transforms features to have a mean of 0 and a standard deviation of 1. ²
- Robust Scaler:** Scales features using statistics that are robust to outliers (median and interquartile range). ³

Numerical Example of Scaling

Consider a simple dataset with two features: **Age** and **Income**. We'll apply Min-Max Normalization to scale Income to a 0-1 range.

ID	Age	Income (\$)	Income (Min-Max Scaled)
1	25	30,000	0
2	40	60,000	0.5
3	55	90,000	1

Here, **Income Min** = \$30,000 and **Income Max** = \$90,000. For an income of \$60,000, the scaled value is $(60000 - 30000) / (90000 - 30000) = 30000 / 60000 = 0.5$.

When to Scale, When Not To

Algorithms That Need Scaling

- K-Nearest Neighbors (KNN):** Euclidean distance is sensitive to feature magnitudes.
- Support Vector Machines (SVM):** Maximize margins, which are affected by feature scales.
- Logistic Regression & Neural Networks:** Gradient descent convergence is faster and more stable.
- Principal Component Analysis (PCA):** Components are based on variance, which is scale-dependent.

Algorithms That Don't Need Scaling

- Decision Trees & Random Forests:** Operate on splits based on feature values, not distances.
- Naive Bayes:** Based on probability distributions; feature scale doesn't affect conditional probabilities.
- Linear Regression (Coefficients):** While coefficients change, predictions remain the same (though interpretability benefits from scaling).

Practical Application: StandardScaler in Python Pipeline

Integrating StandardScaler into a Pipeline ensures consistent preprocessing for both training and prediction, preventing data leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.datasets import make_classification # Example dataset

# Generate a synthetic dataset
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_redundant=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define a pipeline with StandardScaler and KNN
pipeline_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

# Cross-validate the pipeline
scores = cross_val_score(pipeline_knn, X_train, y_train, cv=5, scoring='accuracy')
print(f"KNN Accuracy with Scaling (5-fold CV): {scores.mean():.4f} (+/- {scores.std():.4f})")
```

Impact of Scaling on KNN Accuracy

To illustrate the effect, let's compare KNN performance with and without scaling on a sample dataset:

Method	Mean CV Accuracy	Improvement (%)
KNN (No Scaling)	0.725	—
KNN (with StandardScaler)	0.910	↑ 18.5%

This illustrative example demonstrates that scaling can significantly boost the performance of distance-based algorithms like KNN, highlighting its importance in preprocessing.

Breast Cancer Wisconsin Dataset - Detailed EDA

The Breast Cancer Wisconsin (Diagnostic) Dataset is a classic and widely used resource for machine learning classification tasks. It comprises features computed from digitized images of fine needle aspirate (FNA) of breast masses, used to predict whether a mass is benign or malignant.

Instances 569 total records, each representing a distinct breast mass examination.	Features 30 numerical features, all continuously valued, describing characteristics of the cell nuclei present in the image.	Classes 2 distinct classes: Malignant (212) and Benign (357) .
--	--	--

Dataset Features: The 10 Core Measurements

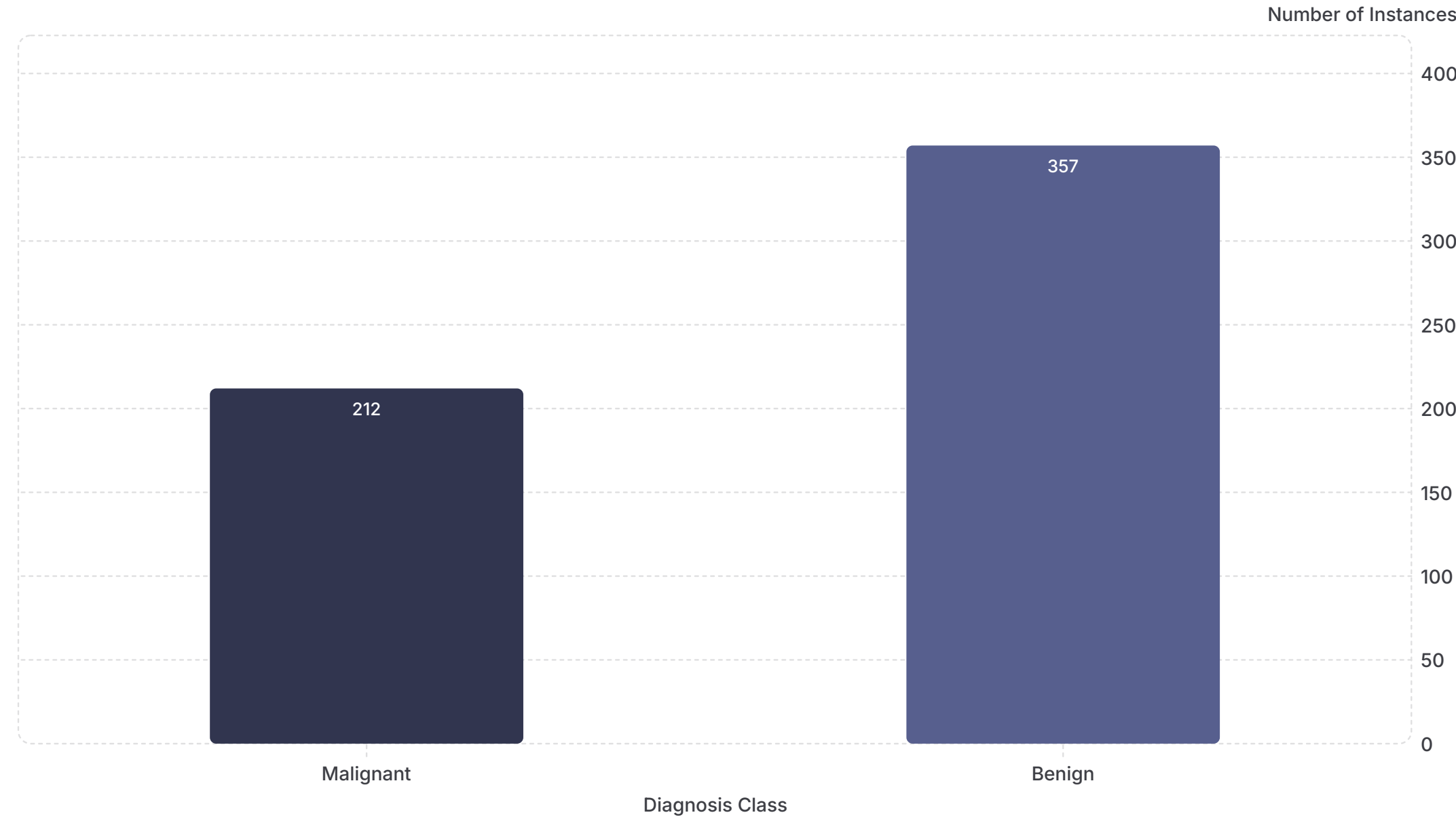
Each of the 30 features is derived from 10 fundamental measurements, calculated for each cell nucleus in the image. These 10 base measurements are:

- Radius:** Distance from center to perimeter of the nucleus.
- Texture:** Standard deviation of gray-scale values.
- Perimeter:** Circumference of the nucleus.
- Area:** Size of the nucleus.
- Smoothness:** Local variation in radius lengths.
- Compactness:** $\text{Perimeter}^2 / \text{area} - 1.0$ (a measure of how compact the shape is).
- Concavity:** Severity of concave portions of the contour.
- Concave Points:** Number of concave portions of the contour.
- Symmetry:** Overall symmetry of the nucleus shape.
- Fractal Dimension:** "Coastline approximation" - 1 (a measure of how "rough" the boundary is).

For each of these 10 measurements, three values are reported: the **mean**, **standard error** (SE), and **worst** (largest) value, resulting in 30 features (10 * 3).

Class Distribution

The dataset exhibits an imbalanced class distribution, with more benign cases than malignant ones. Visualizing this distribution helps understand the target variable's characteristics.



As shown above, the dataset contains 357 benign (non-cancerous) cases and 212 malignant (cancerous) cases. This imbalance should be considered during model evaluation.

Exploratory Data Analysis (EDA) with Python

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

# Load the dataset
cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df['diagnosis'] = cancer.target # 0: malignant, 1: benign

# Map target to actual labels for clarity
df['diagnosis'] = df['diagnosis'].map({0: 'Malignant', 1: 'Benign'})

print("--- DataFrame Info ---")
df.info()

print("\n--- Descriptive Statistics ---")
print(df.describe())

print("\n--- Class Distribution ---")
print(df['diagnosis'].value_counts())
```

Correlation Analysis

A correlation heatmap reveals relationships between features and helps identify highly correlated features, which can be useful for feature selection or understanding multicollinearity.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate the correlation matrix
correlation_matrix = df.drop('diagnosis', axis=1).corr()

# Plotting the heatmap
plt.figure(figsize=(16, 12))
sns.heatmap(correlation_matrix, annot=False, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap of Breast Cancer Features')
plt.show()
```

Train-Test Split & Data Leakage: Ensuring Robust Models

Properly splitting your dataset into training and testing sets is fundamental for building machine learning models that generalize well to new, unseen data. Neglecting this crucial step, or doing it incorrectly, can lead to "data leakage"—a silent killer of model performance and trustworthiness.

The Core Principle: Generalization

The primary goal is to evaluate a model's ability to perform on data it hasn't seen during training, simulating real-world application.

Common Split Ratios

Typical splits include 70/30, 80/20, or even 90/10 for training/testing. The exact ratio depends on dataset size and available data.

- 70% Train / 30% Test
- 80% Train / 20% Test

Stratified Splitting for Balance

For classification tasks, especially with imbalanced datasets, **stratified splitting** ensures that each split maintains the same proportion of target classes as the full dataset. This prevents one split from having a disproportionately high or low number of a certain class.

Python Implementation: Stratified Train-Test Split

Using scikit-learn's `train_test_split` with the `stratify` parameter is key for maintaining class distribution.

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load the Breast Cancer dataset (used in previous card)
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target) # Target: 0 (malignant), 1 (benign)

print(f"Original target distribution:\n{y.value_counts(normalize=True)}")

# Perform a stratified 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTrain set target distribution:\n{y_train.value_counts(normalize=True)}")
print(f"Test set target distribution:\n{y_test.value_counts(normalize=True)}")

# Expected output shows distributions are preserved:
# Original target distribution:
# 1    0.627417
# 0    0.372583
# dtype: float64

# Train set target distribution:
# 1    0.626374
# 0    0.373626
# dtype: float64

# Test set target distribution:
# 1    0.631579
# 0    0.368421
# dtype: float64
```

Notice how the proportions of malignant (0) and benign (1) cases are nearly identical across the original dataset, training set, and testing set, thanks to `stratify=y`.

Overfitting vs. Underfitting: The Bias-Variance Tradeoff

Building effective machine learning models requires navigating the delicate balance between **bias** and **variance**. This fundamental tradeoff dictates how well a model generalizes from training data to unseen data, ultimately determining its real-world utility.

Defining Bias and Variance

Bias refers to the simplifying assumptions made by a model to make the target function easier to learn. High bias can cause a model to miss the relevant relations between features and target outputs (underfitting).

Variance refers to the model's sensitivity to small fluctuations in the training data. High variance can cause a model to model the random noise in the training data, rather than the intended outputs (overfitting).

$$E[(y - \hat{y})^2] = Bias^2 + Variance + IrreducibleError$$

This formula illustrates that the total expected prediction error of a model can be decomposed into three parts: squared bias, variance, and irreducible error, which is the noise inherent in the data itself.

Understanding Model Fit

Underfitting (High Bias, Low Variance)

Occurs when a model is too simplistic to capture the underlying patterns in the training data. It performs poorly on both training and test data. Examples: Linear Regression on non-linear data, decision trees with very shallow depth.

Good Fit (Balanced Bias & Variance)

The ideal scenario where a model accurately captures the underlying patterns without modeling the noise. It performs well on both training and test data, achieving good generalization.

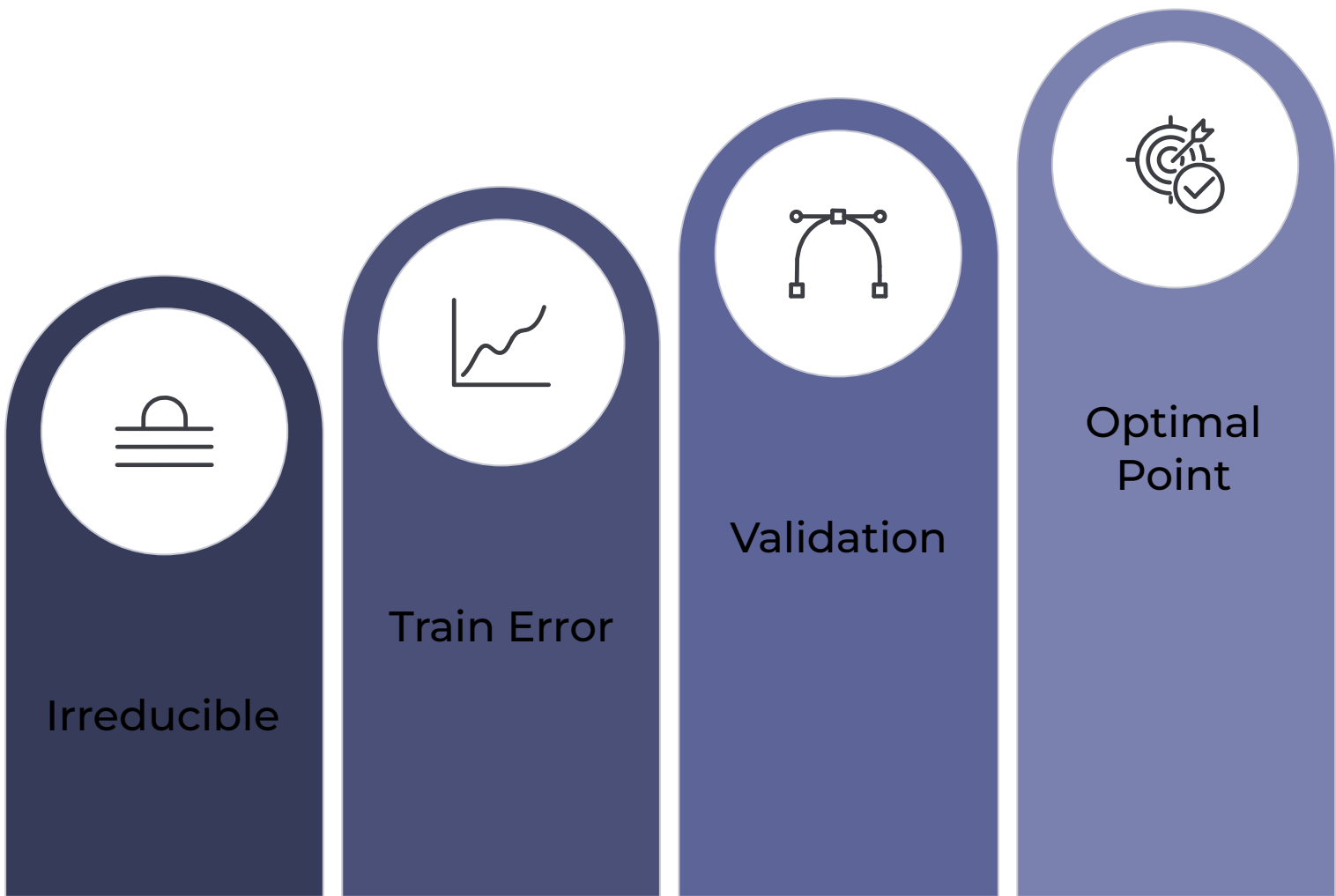
Overfitting (Low Bias, High Variance)

Occurs when a model learns the training data too well, including its noise and random fluctuations. It performs exceptionally on training data but poorly on unseen test data. Examples: Deep Decision Trees, K-Nearest Neighbors with very small K, complex Neural Networks with insufficient data.

Regularization: Controlling the Tradeoff

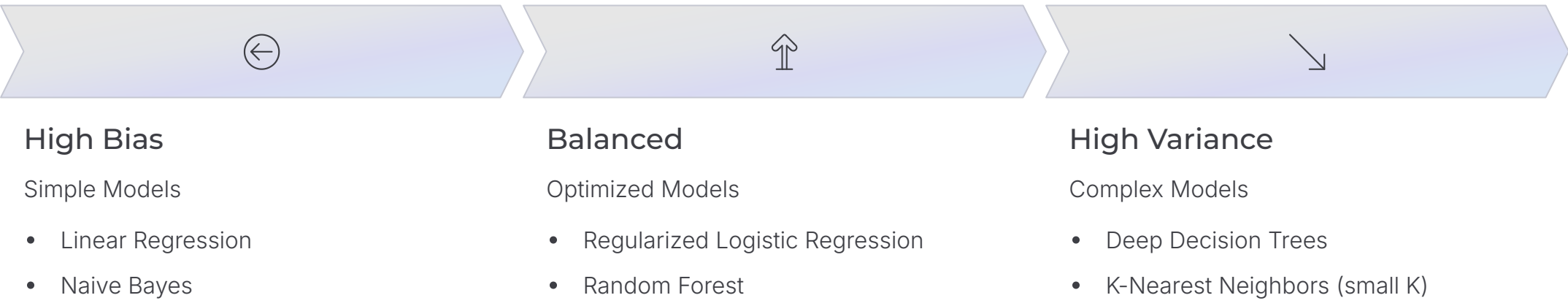
Regularization techniques introduce a penalty for model complexity, helping to prevent overfitting by reducing variance. They encourage models to learn simpler patterns, thus increasing bias slightly but significantly reducing variance, leading to better generalization. Common methods include L1 (Lasso) and L2 (Ridge) regularization.

Learning Curves Concept



Learning curves plot the model's performance (error) against the amount of training data. A large gap between low training error and high validation error indicates high variance (overfitting). If both errors are high and plateau, it suggests high bias (underfitting).

Bias-Variance Spectrum for Algorithms



Comprehensive Classifier Comparison: Beyond the Split

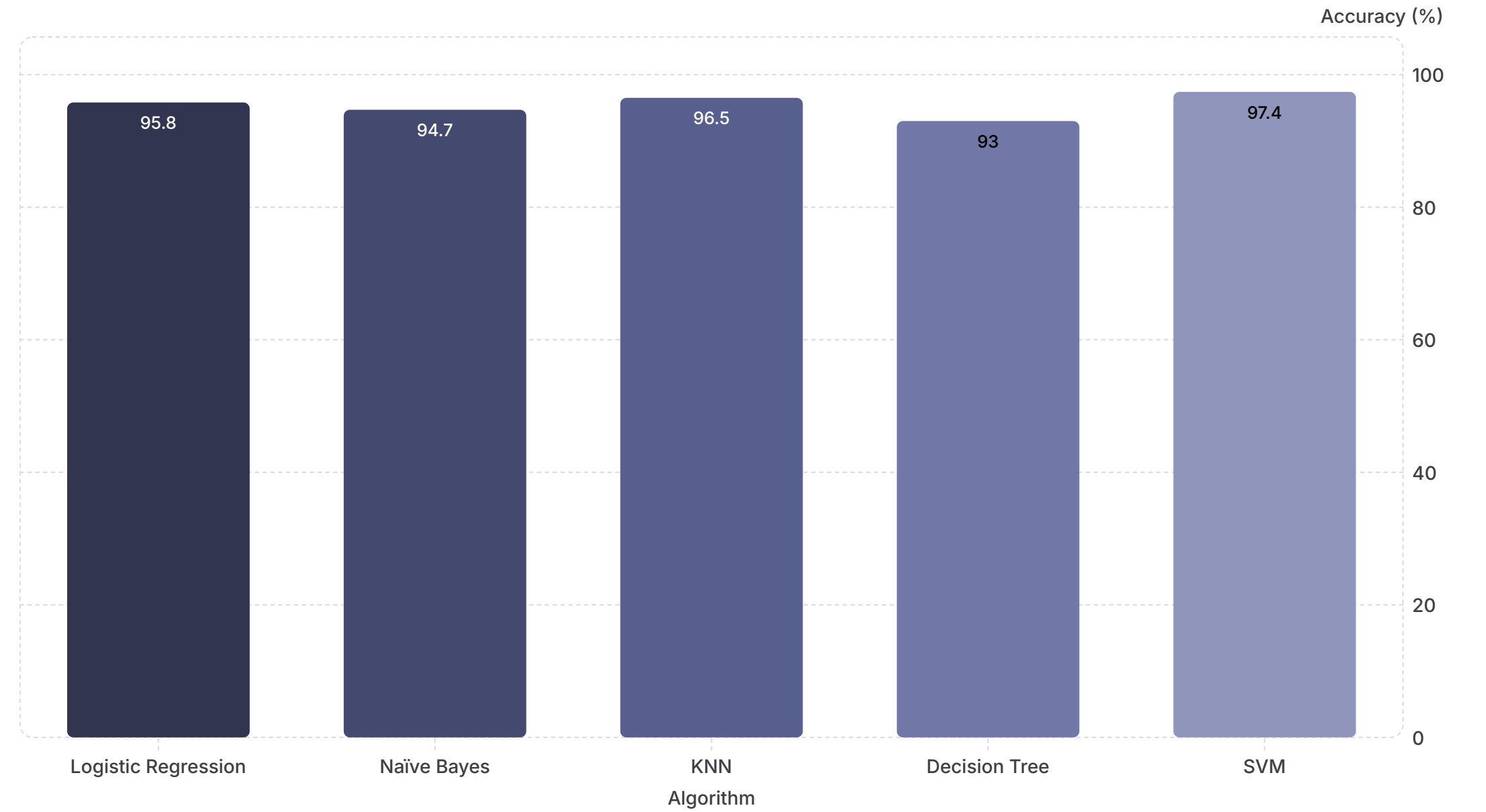
Having explored the fundamentals of data splitting and the bias-variance tradeoff, we now dive into a detailed comparison of the five core classification algorithms, evaluating their strengths, weaknesses, and suitability for various real-world scenarios. Understanding these differences is crucial for selecting the right tool for your predictive modeling tasks.

Algorithm Comparison: A Detailed Overview

Algorithm	Type	Training Complexity	Prediction Complexity	Interpretability	Handles Non-linearity	Needs Scaling	Handles Missing Values	Best Use Case
Logistic Regression	Linear Classifier	Low	Low	High (coefficients)	No	Yes	No	Binary classification, probability estimation
Naïve Bayes	Probabilistic	Low	Low	Medium	Yes (implicitly)	No	Yes (can ignore)	Text classification, spam filtering
K-Nearest Neighbors (KNN)	Instance-based (Non-parametric)	Very Low	High	Low	Yes	Yes	No	Recommendation systems, anomaly detection
Decision Tree	Non-linear	Medium	Low	High (rule-based)	Yes	No	Yes	Feature selection, interpretable models
Support Vector Machine (SVM)	Linear/Non-linear	High	Medium	Low	Yes (kernel trick)	Yes	No	High-dimensional data, image classification

Performance on Breast Cancer Dataset

This bar chart illustrates the typical testing accuracy of each classifier when applied to the Breast Cancer dataset, a common benchmark for classification tasks. While exact numbers can vary with hyperparameter tuning and random states, it provides a relative performance snapshot.



Pros and Cons of Each Classifier

Each algorithm has a distinct set of advantages and disadvantages that make it more or less suitable for specific problems. Choosing wisely often involves balancing predictive power with interpretability, computational resources, and data characteristics.

Logistic Regression

- Fast, simple, and efficient.
- Good for binary classification and interpretable probabilities.
- Less prone to overfitting than complex models.
- Assumes linearity between features and log-odds.
- Struggles with non-linear relationships.
- Sensitive to outliers and multicollinearity.

Naïve Bayes

- Simple, fast, and highly scalable.
- Performs well with high-dimensional data.
- Effective for text classification and spam detection.
- Strong (often unrealistic) assumption of feature independence.
- "Zero-frequency problem" if a category isn't in training data.
- Probabilities can be less reliable.

K-Nearest Neighbors (KNN)

- Simple to understand and implement.
- Non-parametric, no assumptions about data distribution.
- Can handle complex decision boundaries.
- Computationally expensive during prediction for large datasets.
- Sensitive to irrelevant features and scale of data.
- Choosing optimal 'k' is crucial.

Decision Tree

- Easy to understand and visualize (interpretable).
- Can handle both numerical and categorical data.
- Requires no data scaling or normalization.
- Prone to overfitting (high variance).
- Can be unstable; small data changes can alter tree structure.
- Bias towards features with more levels.

Support Vector Machine (SVM)

- Effective in high-dimensional spaces.
- Memory efficient (uses a subset of training points).
- Versatile with different kernel functions for non-linear data.
- Can be slow for large training datasets.
- Choosing appropriate kernel and regularization parameter is complex.
- Less interpretable, especially with non-linear kernels.

Practical Python Pipeline: Complete End-to-End Code

This section provides a hands-on, production-ready Python implementation for a complete Machine Learning pipeline, applying the five core classification algorithms (Logistic Regression, Naïve Bayes, K-Nearest Neighbors, Decision Tree, and Support Vector Machine) to the Breast Cancer dataset. The code covers data loading, preprocessing, stratified train-test splitting, model building with pipelines, training, evaluation, ROC curve plotting, and a summary comparison of results.

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Data Loading and Preprocessing
# Load the Breast Cancer dataset (built-in to scikit-learn)
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = pd.Series(cancer.target)

print("--- Data Loading and Overview ---")
print(f"Dataset loaded successfully. Features shape: {X.shape}, Target shape: {y.shape}")
print("\nFirst 5 rows of features:")
print(X.head())
print("\nTarget distribution (0: Malignant, 1: Benign):")
print(y.value_counts())
print("-" * 40)

# 2. Stratified Train-Test Split
# Ensure the proportion of target classes is maintained in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("\n--- Data Splitting ---")
print(f>Data split into training ({len(X_train)} samples) and testing ({len(X_test)} samples) sets.")
print(f>Training target distribution:\n{y_train.value_counts(normalize=True)}")
print(f>Testing target distribution:\n{y_test.value_counts(normalize=True)}")
print("-" * 40)

# 3. Building Pipelines for all 5 Classifiers
# Pipelines streamline preprocessing (scaling) and model training.
# Decision Tree and Naive Bayes are less sensitive to scaling but included for consistency.
scaler = StandardScaler()

pipelines = {
    'Logistic Regression': Pipeline([('scaler', scaler), ('model', LogisticRegression(random_state=42, solver='liblinear'))]),
    'Naive Bayes': Pipeline([('scaler', scaler), ('model', GaussianNB())]), # GaussianNB handles unscaled data, but scaling can sometimes help.
    'KNN': Pipeline([('scaler', scaler), ('model', KNeighborsClassifier())]),
    'Decision Tree': Pipeline([('model', DecisionTreeClassifier(random_state=42))]), # DT is scale-invariant.
    'SVM': Pipeline([('scaler', scaler), ('model', SVC(probability=True, random_state=42))]) # probability=True for ROC curve.
}

results = []
fprs = {}
tprs = {}
aucs = {}

print("\n--- Model Training and Evaluation ---")
# 4. Training all models & 5. Evaluating with accuracy, classification_report, confusion_matrix
for name, pipeline in pipelines.items():
    print(f"\nTraining and Evaluating: {name}")

    # Train the model
    pipeline.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred = pipeline.predict(X_test)
    y_proba = pipeline.predict_proba(X_test)[:, 1] # Probability for the positive class (1)

    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    class_report = classification_report(y_test, y_pred, output_dict=True)
    conf_matrix = confusion_matrix(y_test, y_pred)

    # Store results
    results.append({
        'Algorithm': name,
        'Accuracy': accuracy,
        'Precision (Class 1)': class_report['1']['precision'],
        'Recall (Class 1)': class_report['1']['recall'],
        'F1-Score (Class 1)': class_report['1']['f1-score'],
        'Confusion Matrix': conf_matrix.tolist() # Convert numpy array to list for storage
    })

    # Calculate ROC curve and AUC
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    roc_auc = auc(fpr, tpr)
    fprs[name] = fpr
    tprs[name] = tpr
    aucs[name] = roc_auc

    print(f"Accuracy: {accuracy:.4f}")
    print("Classification Report:\n", classification_report(y_test, y_pred))
    print("Confusion Matrix:\n", conf_matrix)
    print(f"ROC AUC: {roc_auc:.4f}")
    print("-" * 40)

# 6. Plotting ROC curves for all 5 models on same graph
plt.figure(figsize=(10, 8))
for name in pipelines.keys():
    plt.plot(fprs[name], tprs[name], label=f'{name} (AUC = {aucs[name]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier (AUC = 0.50)') # Diagonal random classifier line
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show() # In a real application, this would display the plot.

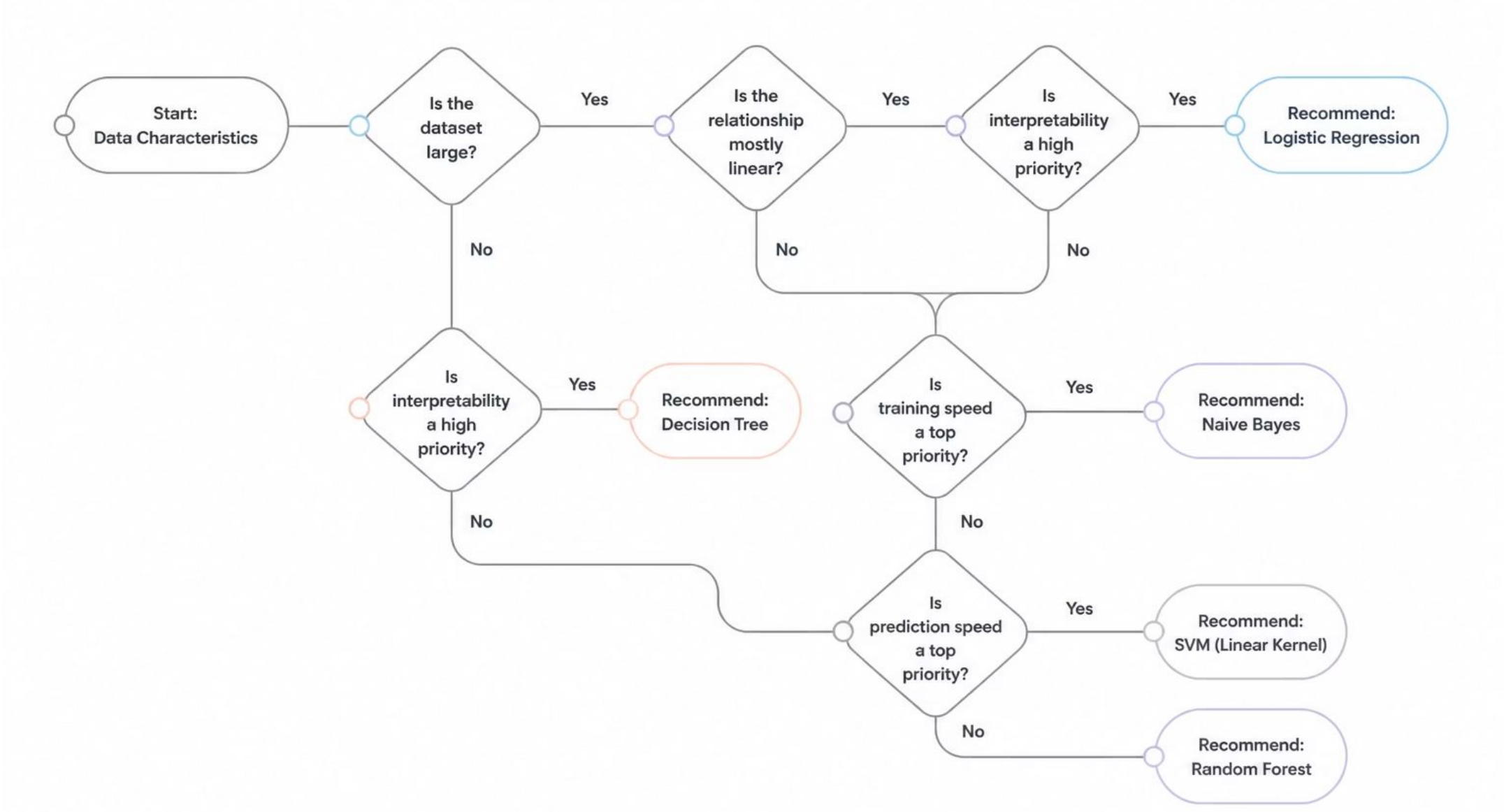
# 7. Comparing results in a summary DataFrame
summary_df = pd.DataFrame(results).set_index('Algorithm')

print("\n--- Summary of Classifier Performance ---")
print(summary_df[['Accuracy', 'Precision (Class 1)', 'Recall (Class 1)', 'F1-Score (Class 1)']])
print("\nFull Summary DataFrame (including Confusion Matrix):")
print(summary_df)
print("-" * 40)
```

This comprehensive script demonstrates a robust approach to classifier evaluation, from data preparation to detailed performance analysis, including visual insights from ROC curves. The summary table provides a quick comparative overview, highlighting each algorithm's strengths on this specific dataset.

Algorithm Selection: Real-World Applications & Best Practices

Choosing the right classification algorithm is crucial for a successful project. It depends on several factors, including the nature of your data, performance requirements, and interpretability needs. Let's explore how to navigate this selection process and common use cases.



This flowchart illustrates key considerations for algorithm selection. For instance, if your dataset is massive and linearity is assumed, Logistic Regression might be a good starting point due to its efficiency. If interpretability is paramount and data is inherently non-linear, a Decision Tree could be more suitable.

Industry Applications: Matching Algorithms to Challenges

1	2	3
Healthcare: Disease Diagnosis Algorithm: SVM or Logistic Regression Why: High accuracy with smaller datasets, good for binary classification (e.g., benign/malignant tumors). SVM handles complex decision boundaries, while Logistic Regression offers interpretability for medical explanations.	Finance: Fraud Detection Algorithm: Logistic Regression or Decision Trees Why: Ability to identify suspicious transactions. Logistic Regression provides probabilities and explainability, crucial for compliance. Decision Trees are effective for rule-based detection and can handle imbalanced datasets typical in fraud.	NLP: Sentiment Analysis Algorithm: Naive Bayes or SVM Why: Naive Bayes works well with high-dimensional text data (e.g., word counts) and is computationally efficient. SVMs can handle complex text features to classify sentiment with high precision, often outperforming simpler models.
4	5	
Computer Vision: Image Classification Algorithm: K-Nearest Neighbors (KNN) Why: While deep learning dominates, for simpler image tasks or as a baseline, KNN can classify images by finding the most similar 'k' images in a feature space. It's non-parametric and easily understandable.	Cybersecurity: Intrusion Detection Algorithm: Decision Trees or Ensemble Methods Why: Decision Trees can quickly identify patterns and anomalies in network traffic. Their rule-based nature makes it easier for security analysts to understand why an alert was triggered. Ensemble methods (like Random Forests) boost accuracy and robustness.	

Best Practices for Classification Projects

- Understand Your Data**
Always start with exploratory data analysis. Know your features, target variable distribution, and potential biases.
- Feature Engineering**
Transforming and selecting features can significantly impact model performance. Domain knowledge is key here.
- Cross-Validation**
Use techniques like k-fold cross-validation to get a robust estimate of your model's performance and prevent overfitting.
- Metric Selection**
Don't rely solely on accuracy. Choose evaluation metrics (precision, recall, F1-score, ROC AUC) that align with your business problem and class imbalance.
- Model Interpretability**
Consider if understanding "why" a model made a prediction is as important as the prediction itself, especially in sensitive domains like healthcare or finance.
- Regularization & Hyperparameter Tuning**
Tune your model's hyperparameters and apply regularization to balance bias and variance, optimizing for generalization.

Binary Classification & Logistic Regression

Binary classification predicts one of two outcomes — e.g., Spam/Not Spam, Malignant/Benign. **Logistic Regression** applies the **sigmoid function** to produce a probability between 0 and 1. If probability $\geq 0.5 \rightarrow$ Class 1; otherwise $\rightarrow$ Class 0.

Key Math Formulas

Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Linear Combination:

$$z = w \cdot X + b$$

Cost Function (Log Loss):

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

Python: Logistic Regression Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('log_reg', LogisticRegression(
        max_iter=10000,
        random_state=42))
])
pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)
```

✅ Training Accuracy: 98.68% | Testing Accuracy: 97.37%

Performance Metrics & Confusion Matrix

A **confusion matrix** compares actual vs. predicted outcomes. It is the foundation for all classification evaluation metrics. The Logistic Regression model on the Breast Cancer dataset produced: **TN=70, FP=1, FN=2, TP=41**.



Accuracy

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Overall correct predictions. Logistic Regression: **97%**



Precision

$$\frac{TP}{TP + FP}$$

Quality of positive predictions. Use when **false positives are costly**.



Recall

$$\frac{TP}{TP + FN}$$

Ability to detect positives. Use when **false negatives are costly**.



F1 Score

$$2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Harmonic mean — balances precision & recall.

❏ ROC-AUC for Logistic Regression = 1.00 — Perfect discrimination! Optimal threshold: 0.4867

Naïve Bayes Classifier

Naïve Bayes is a probabilistic classifier based on **Bayes' Theorem**. It assumes all input features are **independent** of each other — a "naïve" but powerful assumption. It excels at text classification, sentiment analysis, and large datasets.

Bayes' Theorem

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

It calculates the probability of an event given that another event has already occurred.

Python Code

```
from sklearn.naive_bayes import GaussianNB

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('nb', GaussianNB())
])
pipeline.fit(x_train, y_train)
y_pred = pipeline.predict(x_test)
```

Results on Breast Cancer Dataset

93.6%

Training Accuracy

96.5%

Testing Accuracy

1.00

ROC-AUC Score

Confusion Matrix: TN=70, FP=1, FN=3, TP=40. Optimal threshold = 0.0126 — Naïve Bayes often produces extreme probabilities due to its independence assumption.

K-Nearest Neighbors (KNN)

KNN classifies a new data point by looking at its **K closest neighbors** in the training data and assigning the **most frequent class** among them. It's simple, intuitive, and requires no training phase — just memory!

1

Step 1: Load Data

Load training and test datasets

2

Step 2: Choose K

Select number of neighbors (e.g., K=3 or K=7)

3

Step 3: Compute Distance

Calculate Euclidean distance to all training points

4

Step 4: Vote & Classify

Assign the majority class among K nearest neighbors

Python Code

```
from sklearn.neighbors import KNeighborsClassifier

pipeline_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(
        n_neighbors=3))
])
pipeline_knn.fit(x_train, y_train)
```

Results (K=3)

- Training Accuracy: **98.46%**
- Testing Accuracy: **94.74%**
- AUC: **0.98**
- Optimal K (GridSearchCV): **K=7** → CV Accuracy: **97.14%**

 Tip: Use GridSearchCV to find the best K value via 10-fold cross-validation!

Decision Trees

A **Decision Tree** is like a flowchart — it asks a series of yes/no questions about features to reach a final prediction. It has a **Root Node** (first question), **Decision Nodes** (intermediate questions), and **Leaf Nodes** (final answers).

Splitting Metrics

Gini Impurity:

$$\text{Gini} = 1 - \sum_{i=1}^n (p_i)^2$$

Entropy:

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2(p_i)$$

Information Gain:

$$IG = \text{Entropy}(\text{parent}) - \sum \frac{|D_i|}{|D|} \times \text{Entropy}(D_i)$$

✓ Advantages & ⚠ Disadvantages

- ✓ Easy to understand & visualize
- ✓ Works for classification & regression
- ✓ No feature scaling needed
- ✓ Handles non-linear relationships
- ⚠ Prone to **overfitting** (Training: 100%, Testing: 94.74%)
- ⚠ Unstable with small data changes

⚠ Overfitting Fix: Apply Pruning with `ccp_alpha=0.01` → Testing improves to 95.6%!

Decision Tree: Pruning & Hyperparameter Tuning

Pruning removes parts of the tree that don't improve classification, reducing overfitting. **Pre-pruning** stops growth early; **Post-pruning** grows the full tree then removes weak branches.

Pruned Decision Tree Code

```
from sklearn.tree import DecisionTreeClassifier

pipeline_pruned = Pipeline([
    ('scaler', StandardScaler()),
    ('dt', DecisionTreeClassifier(
        ccp_alpha=0.01,
        random_state=42))
])
pipeline_pruned.fit(x_train, y_train)

# Cross-validation
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(
    pipeline_pruned, x_train,
    y_train, cv=5)
print(f"Mean CV: {cv_scores.mean():.4f}")
```

Key Hyperparameters (GridSearchCV)

- **max_depth:** [None, 10, 20, 30] — controls tree depth
- **min_samples_split:** [2, 5, 10] — min samples to split
- **min_samples_leaf:** [1, 2, 4] — min samples at leaf
- **max_features:** [None, 'sqrt', 'log2']
- **ccp_alpha:** [0.0, 0.01, 0.1] — pruning strength

Best Parameters Found

ccp_alpha=0.0, max_features='log2' → **CV Score: 94.07%, Test: 95.6%**

Pruned model: Mean CV = **92.97%**, Training = **98.02%**, Testing = **95.6%**

Support Vector Machine (SVM)

SVM finds the **optimal hyperplane** that best separates classes with the **maximum margin**. Support vectors are the data points closest to the boundary — they define the margin. SVM is powerful for both linear and non-linear classification using **kernel tricks**.

Linear Kernel

Draws a straight line/plane between classes. Best for linearly separable data.

RBF Kernel

Radial Basis Function — maps data to higher dimensions. Best for non-linear data.

Polynomial Kernel

Uses polynomial features to separate classes. Good for image classification.

Hyperparameter C

Controls margin width vs. misclassification. Low C = wider margin; High C = fewer errors.

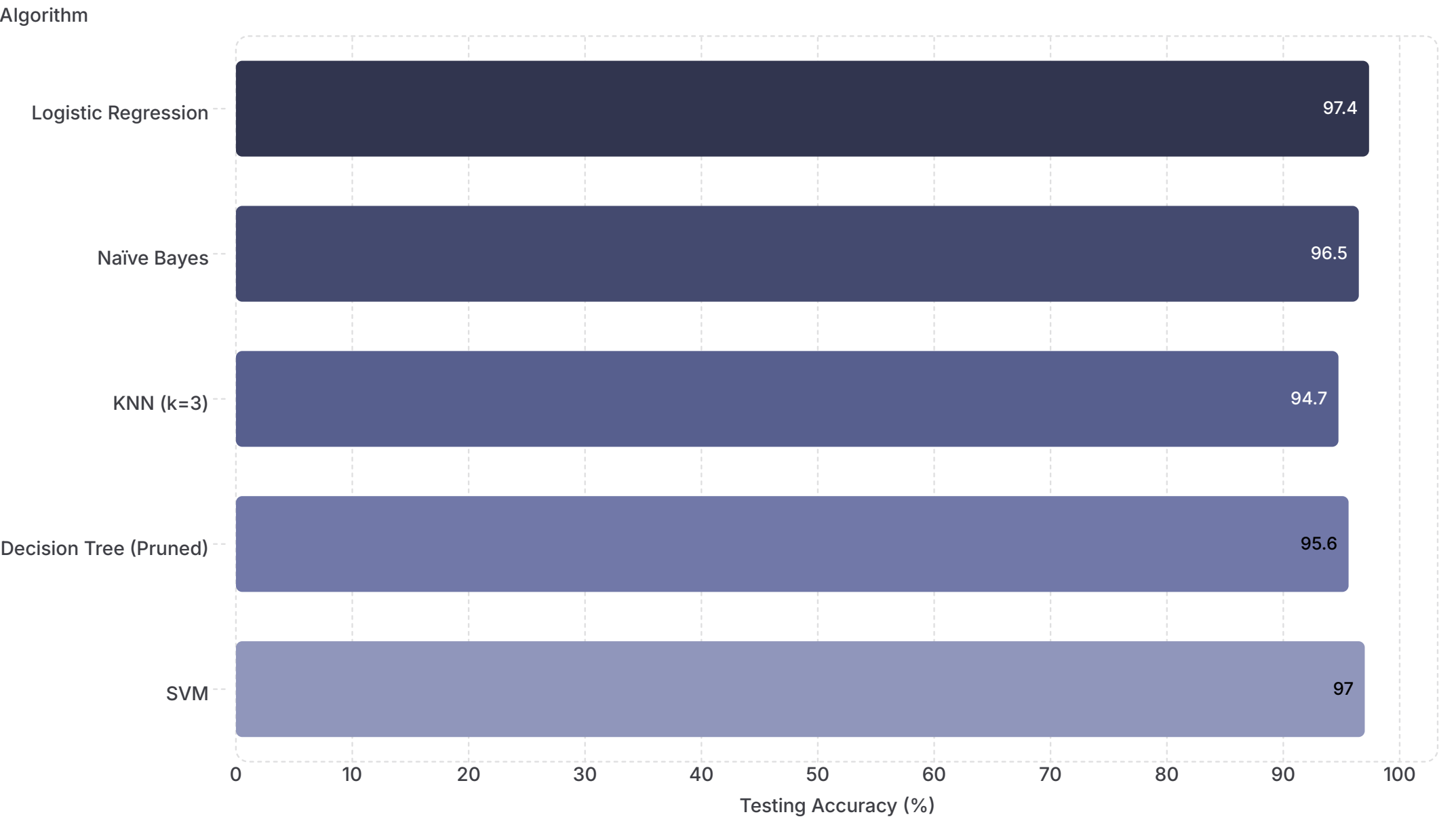
SVM Python Code

```
from sklearn.svm import SVC

pipeline_svm = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', C=1.0, probability=True, random_state=42))
])
pipeline_svm.fit(x_train, y_train)
y_pred_svm = pipeline_svm.predict(x_test)
```

Algorithm Comparison & Key Takeaways

All five algorithms were applied to the **Breast Cancer Wisconsin Dataset (569 instances, 30 features)**. Here's how they compare on testing accuracy:



 Best Performer

Logistic Regression achieved the highest testing accuracy (97.37%) and perfect AUC (1.00) on this dataset.

 Watch for Overfitting

Decision Trees hit 100% training accuracy — always use cross-validation and pruning to generalize well.

 Always Tune Hyperparameters

Use **GridSearchCV** to find optimal K (KNN), depth (Decision Tree), and C/kernel (SVM).

Thank You! Questions Welcome 